

Bridging the Gap: A Comparative Analysis of Classical Heuristics vs. Deep Reinforcement Learning in Algorithmic Trading

Authors:

Kunal Sharma (2022610)
Sarthak Kandpal (2022453)

Independent Study Project

Supervised by: Professor Pankaj Vajpayee

December 13, 2025

Abstract

This comprehensive study explores the efficacy of diverse algorithmic trading strategies, ranging from classical technical analysis to state-of-the-art Deep Reinforcement Learning (DRL). We implement and benchmark six distinct approaches: (1) A Buy and Hold benchmark, (2) A Cross-Sectional Momentum strategy based on Jegadeesh & Titman (1993), (3) A "Perfect Order" Trend Following strategy utilizing multiple Moving Averages, (4) Statistical Arbitrage methods including Pairs Trading and the Reservation Price Min-Max (RPMM) algorithm, and (5) A Trading Deep Q-Network (TDQN) agent trained on artificially augmented financial data.

To ensure a rigorous comparison, a custom Event-Driven Backtesting Engine was developed in Python, supporting multi-asset portfolios, dynamic transaction costs, and realistic risk management protocols. All strategies were evaluated on a unified dataset comprising major US equities (AAPL, GOOGL, MSFT) over a 9-year period (2015-2023), totaling 2,264 trading days.

Our results demonstrate that the TDQN agent achieved the highest total return (653.5%) and risk-adjusted return (Sharpe Ratio of 1.02), outperforming the Buy and Hold benchmark (612.4%, Sharpe 1.01). However, the statistical significance of this outperformance is marginal ($p=0.078$), suggesting that while AI provides an edge, the improvement requires more data to confirm conclusively.

Among classical strategies, the Perfect Order trend-following approach demonstrated excellent risk management with the lowest drawdown among directional strategies (-23.5%), while the RPMM algorithm achieved the best capital preservation with only -16.9% maximum drawdown. The Momentum strategy, despite its strong theoretical foundation, underperformed the benchmark due to high turnover costs and the January effect filter.

This report details the theoretical foundations, implementation architecture, and experimental results of this comparative laboratory, concluding with a critical analysis of the "Human Logic vs. AI Logic" paradigm in modern finance. We find that while AI provides promising results, classical strategies remain valuable tools for risk management and portfolio diversification.

Keywords: Algorithmic Trading, Deep Reinforcement Learning, Q-Learning, Momentum, Technical Analysis, Statistical Arbitrage, Backtesting, Sharpe Ratio

Contents

Abstract	1
1 Introduction	8
1.1 Background of Algorithmic Trading	8
1.1.1 Historical Evolution	8
1.1.2 The Two Schools of Thought	8
1.2 Problem Statement	9
1.2.1 The Research Gap	9
1.2.2 Significance of the Study	10
1.3 Objectives of the Study	10
1.3.1 Primary Objectives	10
1.3.2 Secondary Objectives	10
1.4 Research Hypotheses	11
1.5 Scope and Limitations	11
1.5.1 Scope	11
1.5.2 Limitations	11
1.6 Structure of the Report	12
1.7 Key Definitions	12
2 Literature Review	14
2.1 The Evolution of Market Efficiency	14
2.1.1 Forms of Market Efficiency	14
2.1.2 Challenges to EMH	14
2.2 Deep Reinforcement Learning (Théate & Ernst, 2020)	15
2.2.1 Overview	15
2.2.2 Theoretical Foundation	15
2.2.3 The TDQN Architecture	15
2.2.4 Key Innovations	16
2.2.5 Results from the Original Paper	16
2.3 Statistical Arbitrage and RPMM (Hägg, 2023)	16
2.3.1 Overview	16
2.3.2 Theoretical Foundation: Mean Reversion	17
2.3.3 Pairs Trading Methodology	17
2.3.4 Reservation Price Min-Max (RPMM)	18
2.3.5 Advantages and Limitations	18
2.4 Momentum (Jegadeesh & Titman, 1993)	18
2.4.1 Overview	18
2.4.2 The Discovery	19

2.4.3	The J/K Framework	19
2.4.4	The Skip-Week Filter	19
2.4.5	Behavioral Explanations	19
2.4.6	The January Effect	20
2.5	Technical Analysis (Perfect Order)	20
2.5.1	Overview	20
2.5.2	Moving Average Theory	20
2.5.3	The Perfect Order Setup	20
2.5.4	The ADX Confirmation	21
2.5.5	Risk Management Focus	21
2.6	Literature Synthesis	21
3	Methodology and Architecture	22
3.1	The Event-Driven Backtesting Engine	22
3.1.1	Why Event-Driven?	22
3.1.2	Comparison with Vectorized Backtesting	22
3.1.3	System Architecture	23
3.2	Order Execution Model	24
3.2.1	Execution Assumptions	24
3.2.2	Order Types	25
3.2.3	Commission Calculation	25
3.3	Risk Management Module	25
3.3.1	The 2% Rule	25
3.3.2	ATR-Based Stop Loss	26
3.3.3	Volatility-Adjusted Position Sizing	26
3.4	Performance Metrics	26
3.4.1	Total Return	27
3.4.2	Compound Annual Growth Rate (CAGR)	27
3.4.3	Sharpe Ratio	27
3.4.4	Maximum Drawdown	27
3.4.5	Geometric Mean Return	28
3.4.6	Win Rate	28
3.4.7	Annualized Volatility	28
3.5	Data Description	28
3.5.1	Data Sources	28
3.5.2	Selected Assets	28
3.5.3	Time Period	28
3.5.4	Data Quality	29
3.6	Experimental Design	29
3.6.1	Backtesting Parameters	29
3.6.2	Strategy Configurations	29
3.6.3	Reproducibility	29
4	Implementation of Classical Strategies	31
4.1	Momentum Strategy Implementation	31
4.1.1	Theoretical Basis	31
4.1.2	Algorithm Design	31
4.1.3	Implementation Details	32

4.1.4	The January Effect Filter	32
4.1.5	Signal Flow Diagram	33
4.2	Perfect Order Strategy Implementation	33
4.2.1	The "Perfect Order" Setup	33
4.2.2	Mathematical Definition	33
4.2.3	Entry and Exit Rules	34
4.2.4	Position Sizing	34
4.2.5	Technical Indicators Used	34
4.2.6	Implementation Considerations	35
4.3	Statistical Arbitrage: Pairs Trading and RPMM	35
4.3.1	Pairs Trading	35
4.3.2	Reservation Price Min-Max (RPMM)	36
4.4	Strategy Comparison Summary	37
4.5	Risk Considerations	38
4.5.1	Momentum Risks	38
4.5.2	Perfect Order Risks	38
4.5.3	Statistical Arbitrage Risks	38
5	Implementation of Deep Reinforcement Learning	39
5.1	Reinforcement Learning Theoretical Framework	39
5.1.1	The RL Paradigm	39
5.1.2	Markov Decision Process (MDP)	39
5.1.3	Value Functions	40
5.1.4	Bellman Equations	40
5.1.5	Q-Learning	40
5.2	Deep Q-Networks (DQN)	40
5.2.1	Motivation	40
5.2.2	DQN Architecture	41
5.2.3	Training Algorithm	41
5.2.4	Key Innovations in DQN	41
5.3	The TDQN Architecture	42
5.3.1	State Space Design	42
5.3.2	Action Space	43
5.3.3	Reward Function Design	43
5.3.4	Transaction Cost Integration	43
5.4	Data Augmentation: Artificial Trajectory Generation	43
5.4.1	The Data Scarcity Problem	44
5.4.2	Solution: Trajectory Slicing	44
5.4.3	Benefits of Data Augmentation	44
5.5	Training Process	45
5.5.1	Environment Setup	45
5.5.2	Training Configuration	45
5.5.3	Training Procedure	45
5.5.4	Convergence Monitoring	46
5.6	Inference (Trading) Mode	46
5.7	Comparison: Classical vs AI Approaches	47

6	Experimental Setup and Results	48
6.1	Experimental Setup	48
6.1.1	Data Description	48
6.1.2	Data Split	48
6.1.3	Backtesting Parameters	48
6.2	Comparative Performance Analysis	49
6.2.1	Summary Results	49
6.2.2	Performance Metrics Visualization	49
6.2.3	Key Observations	50
6.3	Detailed Analysis by Strategy	50
6.3.1	Buy & Hold (Benchmark)	50
6.3.2	Momentum Strategy (Jegadeesh & Titman, 1993)	50
6.3.3	Perfect Order Strategy (Technical Analysis)	51
6.3.4	TDQN AI (Théate & Ernst, 2020)	51
6.3.5	RPMM (Hägg, 2023)	51
6.3.6	Pairs Trading (Hägg, 2023)	52
6.4	Equity Curves	52
6.5	Risk Analysis	53
6.5.1	Risk-Return Profile	53
6.5.2	Drawdown Analysis	54
6.5.3	Volatility Analysis	54
6.5.4	Tail Risk	54
6.6	Statistical Significance	55
6.6.1	T-Test for Excess Returns	55
6.6.2	Sharpe Ratio Confidence Intervals	55
6.7	Market Regime Analysis	56
6.7.1	Performance by Market Regime	56
6.7.2	Correlation Analysis	56
6.8	Comparison with Roadmap Expectations	56
6.9	Summary of Findings	57
7	Conclusion and Future Work	58
7.1	Summary of Findings	58
7.1.1	Primary Findings	58
7.2	Complexity vs. Robustness Trade-off	58
7.2.1	The Case for Simple Strategies	58
7.2.2	The Case for AI Strategies	59
7.2.3	Recommendation	59
7.3	The "No Free Lunch" Theorem	59
7.4	The Value of Baselines	59
7.5	Lessons Learned	60
7.5.1	Technical Lessons	60
7.5.2	Strategic Lessons	60
7.6	Future Work	60
7.6.1	Model Improvements	60
7.6.2	Feature Engineering	60
7.6.3	Risk Management	61
7.6.4	Practical Implementation	61

7.7	Final Thoughts	61
7.8	Acknowledgments	61
7.9	Declarations	62
A	Source Code Listings	63
A.1	Project Structure	63
A.2	Base Strategy Interface	64
A.3	Backtester Engine	64
A.4	Performance Metrics Module	68
A.5	Buy and Hold Strategy (Benchmark)	71
A.6	Momentum Strategy (Jegadeesh & Titman, 1993)	72
A.7	Perfect Order Strategy (Technical Analysis)	75
A.8	Pairs Trading Strategy (Hägg, 2023)	78
A.9	RPMM Strategy (Hägg, 2023)	82
A.10	TDQN Agent (Théate & Ernst, 2020)	83
A.11	Data Augmentation Module	86
A.12	Main Comparison Script	87
A.13	Data Download Script	89
A.14	Dependencies	90
A.15	Usage Instructions	90
	A.15.1 Installation	90
	A.15.2 Running the Backtest	90
	A.15.3 Training the TDQN Agent	91
A.16	Configuration Parameters	91
	A.16.1 Strategy Parameters	91
	A.16.2 Backtester Parameters	91
A.17	Output Files	92
	A.17.1 Comparison Metrics CSV	92
	A.17.2 Equity Curves Plot	92

List of Figures

4.1	Momentum Strategy Flow	33
5.1	DQN Network Architecture	41
6.1	Performance Metrics Comparison: Total Return, Sharpe Ratio, Maximum Drawdown, and Volatility for all strategies. TDQN AI (bright blue) achieves the highest return and Sharpe ratio, while RPMM shows the lowest drawdown.	49
6.2	Strategy Comparison: Equity Curves (2015-2024). The TDQN AI agent (thick blue line) and Buy & Hold (dashed gray) show the highest growth. Shaded regions indicate the COVID crash (red) and 2022 bear market (orange).	52
6.3	Risk-Return Profile of Trading Strategies. The ideal position is upper-left (high return, low risk). TDQN achieves the highest return, while RPMM occupies the conservative lower-left quadrant.	53
6.4	Drawdown Analysis Over Time. RPMM (blue) shows the shallowest drawdowns, while directional strategies experienced significant losses during the COVID crash and 2022 bear market.	54

List of Tables

2.1	Comparison of Strategy Characteristics	21
3.1	Vectorized vs Event-Driven Backtesting	22
3.2	Backtesting Parameters	29
3.3	Strategy Configurations	29
4.1	Pairs Trading Signals	36
4.2	Classical Strategy Summary	37
5.1	TDQN Training Hyperparameters	45
5.2	Classical vs AI Strategy Comparison	47
6.1	Backtesting Configuration	48
6.2	Comprehensive Performance Metrics (2015-2023)	49
6.3	Drawdown Statistics	54
6.4	Sharpe Ratio Confidence Intervals	55
6.5	Performance by Market Regime (qualitative)	56
6.6	Strategy Return Correlations	56
6.7	Expected vs Actual Returns	57
7.1	Strategy Recommendations by Investor Profile	59
A.1	Default Strategy Parameters	91
A.2	Backtester Configuration	91

Chapter 1

Introduction

1.1 Background of Algorithmic Trading

The financial markets have undergone a paradigm shift over the last few decades, transitioning from open-outcry trading pits to highly sophisticated electronic networks. Algorithms now account for the vast majority of trading volume in global equity markets—estimated at over 70% of all US equity trades as of 2023. This shift has been driven by the dual forces of technological advancement and the increasing complexity of financial instruments.

In the early days, "Algorithmic Trading" simply referred to the automation of order execution—slicing large orders into smaller chunks to minimize market impact (e.g., VWAP or TWAP algorithms). However, the definition has expanded to encompass "Proprietary Trading" strategies where the algorithm itself decides *what* to buy and *when*, based on pre-defined logic or predictive models.

1.1.1 Historical Evolution

The evolution of algorithmic trading can be traced through several key phases:

1970s-1980s: The Birth of Electronic Trading The introduction of NASDAQ in 1971 marked the beginning of electronic trading. The New York Stock Exchange followed with its Designated Order Turnaround (DOT) system in 1976, enabling electronic routing of orders.

1990s: The Rise of ECNs and Day Trading Electronic Communication Networks (ECNs) emerged, providing alternative venues for trading. This era saw the democratization of market access, with retail traders gaining direct market access through online brokers.

2000s: High-Frequency Trading Emerges The decimalization of stock prices in 2001 (moving from fractions to pennies) reduced spreads and opened opportunities for high-frequency trading (HFT). Firms began competing on microsecond execution speeds.

2010s-Present: Machine Learning and AI The current era is characterized by the application of machine learning, deep learning, and reinforcement learning to trading strategies. This study focuses on this cutting-edge frontier.

1.1.2 The Two Schools of Thought

Two distinct schools of thought have emerged in this domain:

1. **Classical Quantitative Finance:** Relying on statistical properties, econometrics, and fixed mathematical rules (e.g., Mean Reversion, Momentum). These strategies are transparent, explainable, and grounded in decades of academic research. Key advantages include:
 - Interpretability and ease of debugging
 - Strong theoretical foundations
 - Lower computational requirements
 - Established regulatory compliance frameworks
2. **Modern Artificial Intelligence:** Utilizing Machine Learning (ML) and Deep Reinforcement Learning (DRL) to uncover non-linear patterns in data that are invisible to human analysts. These models are often criticized as "black boxes" but promise adaptability that static rules cannot match. Potential advantages include:
 - Ability to capture complex, non-linear relationships
 - Automatic adaptation to changing market conditions
 - No need for explicit feature engineering
 - Potential to discover novel trading patterns

1.2 Problem Statement

Despite the hype surrounding AI in finance, there remains a significant gap between theoretical promise and practical viability for retail or independent traders. Deep Reinforcement Learning (DRL) papers often show spectacular results in controlled environments, but are they robust? Can a complex Neural Network actually outperform a simple, proven heuristic like "Buy Winners"?

1.2.1 The Research Gap

Several key questions remain unanswered in the academic literature:

- **Reproducibility:** Many published DRL trading results are difficult to reproduce due to incomplete methodology descriptions.
- **Fair Comparison:** Studies often compare AI strategies against weak baselines, making it difficult to assess true improvements.
- **Transaction Costs:** Many backtests ignore realistic transaction costs, leading to overly optimistic results.
- **Market Regime Sensitivity:** How do strategies perform across different market conditions (bull, bear, sideways)?
- **Practical Implementation:** What infrastructure is required to deploy these strategies in practice?

This study addresses the central question: **Does the complexity of Deep Reinforcement Learning provide a statistically significant edge over classical, rule-based algorithmic trading strategies in the context of US Equities?**

1.2.2 Significance of the Study

This research contributes to the field in several important ways:

1. Provides a comprehensive, reproducible comparison of classical and AI-based strategies
2. Develops a reusable backtesting framework for future research
3. Offers practical insights for traders considering AI adoption
4. Bridges the gap between academic theory and industry practice

1.3 Objectives of the Study

The primary objective of this project is to construct a comparative laboratory to rigorously test "Human Logic" against "AI Logic".

1.3.1 Primary Objectives

1. To design and implement a robust, event-driven backtesting engine capable of simulating multi-asset portfolios with transaction costs.
2. To reproduce three distinct "Classical" trading strategies from fundamental academic literature:
 - **Momentum:** Based on Jegadeesh & Titman (1993).
 - **Trend Following:** The "Perfect Order" technical setup.
 - **Statistical Arbitrage:** Pairs Trading and the RPMM algorithm.
3. To implement a state-of-the-art Deep Reinforcement Learning agent (Trading Deep Q-Network or TDQN) as proposed by Théate & Ernst (2020), utilizing Data Augmentation to overcome data scarcity.
4. To conduct a side-by-side performance analysis using industry-standard metrics (Sharpe Ratio, Maximum Drawdown, CAGR) to evaluate risk-adjusted returns.

1.3.2 Secondary Objectives

1. To analyze the behavior of each strategy across different market regimes
2. To identify the strengths and weaknesses of each approach
3. To provide recommendations for strategy selection based on investor risk profiles
4. To document best practices for algorithmic trading system development

1.4 Research Hypotheses

Based on the literature review and preliminary analysis, we formulate the following hypotheses:

H1: The TDQN agent will achieve a higher risk-adjusted return (Sharpe Ratio) than classical strategies over the test period.

H2: Classical strategies will demonstrate lower maximum drawdown due to their rule-based risk management.

H3: The Momentum strategy will outperform Buy & Hold during trending market regimes.

H4: Statistical arbitrage strategies (Pairs Trading, RPM) will show the highest win rate but lowest total return.

1.5 Scope and Limitations

This study focuses on daily timeframe trading of major US Equities (S&P 500 constituents) and selected technology stocks (AAPL, GOOGL, MSFT). The choice of daily data reduces the impact of microstructure noise and high-frequency execution latency, making it suitable for an independent study.

1.5.1 Scope

- **Asset Classes:** US Equities (Technology Sector)
- **Time Horizon:** Daily trading (no intraday)
- **Period:** 2015-2024 (9 years of data)
- **Strategies:** 6 distinct strategies including a benchmark
- **Metrics:** Total Return, Sharpe Ratio, Max Drawdown, Win Rate, CAGR

1.5.2 Limitations

Limitations:

- **Slippage:** We assume trade execution at the "Close" price of the signal day. In reality, slippage would occur, particularly for larger orders.
- **Market Impact:** The simulation assumes our trades do not affect the market price (valid for small capital). This assumption breaks down for institutional-sized orders.
- **Data Quality:** We rely on free data sources which may have survivorship bias (though we attempt to mitigate this by selecting current major constituents).
- **Regime Changes:** Past performance does not guarantee future results. Market regimes evolve over time.
- **Transaction Costs:** We use a fixed 0.1% commission rate; actual costs vary by broker and order size.

- **Shorting Constraints:** Our implementation has limited support for short selling, particularly in the Pairs Trading strategy.
- **Tax Implications:** Tax effects are not considered, which can significantly impact net returns.

1.6 Structure of the Report

This report is organized as follows:

- **Chapter 2** provides an extensive review of the literature, covering the four key papers that form the theoretical foundation of this study.
- **Chapter 3** details the Methodology and the Architecture of the Backtesting Engine, including system design decisions and implementation details.
- **Chapter 4** describes the implementation of the three Classical Strategies (Momentum, Perfect Order, Statistical Arbitrage).
- **Chapter 5** delves into the AI implementation, covering the DQN architecture, training process, and data augmentation techniques.
- **Chapter 6** presents the Experimental Results and Comparative Analysis, including performance tables, equity curves, and statistical tests.
- **Chapter 7** concludes the study with key findings, practical recommendations, and suggestions for future research.
- **Appendix** contains the complete source code listings for all implemented components.

1.7 Key Definitions

For clarity, we define the following terms used throughout this report:

Algorithmic Trading The use of computer algorithms to automatically generate and execute trading decisions.

Backtesting The process of testing a trading strategy on historical data to evaluate its performance.

Sharpe Ratio A measure of risk-adjusted return, calculated as the ratio of excess return to standard deviation.

Maximum Drawdown The largest peak-to-trough decline in portfolio value over a period.

CAGR Compound Annual Growth Rate; the annualized rate of return accounting for compounding.

Alpha The excess return of a strategy relative to a benchmark.

Beta The sensitivity of a strategy's returns to market movements.

Reinforcement Learning A machine learning paradigm where an agent learns to make decisions by interacting with an environment and receiving rewards.

Deep Q-Network (DQN) A neural network architecture that approximates the Q-value function in reinforcement learning.

Chapter 2

Literature Review

2.1 The Evolution of Market Efficiency

The foundation of modern financial theory rests on the Efficient Market Hypothesis (EMH), proposed by Eugene Fama in the 1960s. EMH posits that asset prices fully reflect all available information, implying that it is impossible to consistently "beat the market" on a risk-adjusted basis.

2.1.1 Forms of Market Efficiency

Fama (1970) defined three forms of market efficiency:

1. **Weak Form:** Prices reflect all historical price and volume information. Technical analysis should not generate excess returns.
2. **Semi-Strong Form:** Prices reflect all publicly available information. Neither technical nor fundamental analysis should generate excess returns.
3. **Strong Form:** Prices reflect all information, including private/insider information. No investor can consistently outperform.

2.1.2 Challenges to EMH

However, the field of Behavioral Finance has challenged this view, suggesting that cognitive biases create persistent anomalies:

- **Overreaction:** Investors overreact to news, causing prices to overshoot fair value.
- **Underreaction:** Investors underreact to earnings announcements, creating post-earnings drift.
- **Herding:** Investors follow the crowd, amplifying market trends.
- **Loss Aversion:** Investors feel losses more intensely than gains, affecting selling decisions.
- **Anchoring:** Investors anchor to arbitrary reference points when making decisions.

It is these anomalies that Algorithmic Traders seek to exploit. Our study focuses on four specific papers that target different market inefficiencies.

2.2 Deep Reinforcement Learning (Théate & Ernst, 2020)

2.2.1 Overview

The core AI strategy in this project is based on the paper "*An Application of Deep Reinforcement Learning to Algorithmic Trading*" by Théate and Ernst (2020). This work represents a significant departure from traditional supervised learning (predicting tomorrow's price). Instead, it frames trading as a control problem.

2.2.2 Theoretical Foundation

The authors observe that traditional ML approaches to trading suffer from a fundamental mismatch:

- Supervised learning predicts prices, but price prediction accuracy doesn't directly translate to trading profits.
- The trading problem is inherently sequential—today's decision affects tomorrow's opportunities.
- Transaction costs create path dependency—it matters *how* you reach a position, not just *that* you're in it.

Reinforcement Learning naturally handles these challenges by:

1. Directly optimizing for cumulative reward (trading profit)
2. Modeling the sequential nature of trading decisions
3. Incorporating transaction costs into the reward function

2.2.3 The TDQN Architecture

The authors propose the **Trading Deep Q-Network (TDQN)**. Unlike standard DQN applications (like Atari games), the trading environment has unique challenges:

- **Stochasticity:** Markets are noisier than video games. The same "action" can lead to vastly different outcomes.
- **Non-Stationarity:** Market regimes change over time. A strategy that works in 2019 may fail in 2020.
- **Data Scarcity:** We only have one history of the stock market. Unlike games, we can't generate unlimited training data.
- **Delayed Rewards:** The impact of a trading decision may not be apparent for days or weeks.

2.2.4 Key Innovations

The paper introduces several innovations to address these challenges:

1. Artificial Trajectory Generation This data augmentation technique addresses data scarcity. By slicing the long historical time-series into shorter, overlapping windows, the agent can be trained on thousands of "pseudo-episodes," effectively preventing overfitting to a specific timeline.

The algorithm works as follows:

1. Given a historical price series of length T
2. Define a window size W (e.g., 252 trading days)
3. Define a step size S (e.g., 60 days)
4. Generate $(T - W)/S$ overlapping trajectories
5. During training, randomly sample from these trajectories

2. Modified Reward Function Instead of using raw profit, the authors propose using risk-adjusted returns:

$$r_t = \frac{V_t - V_{t-1}}{V_{t-1}} - \lambda \cdot |r_t| \quad (2.1)$$

where λ is a penalty term for volatility.

3. Experience Replay with Prioritization The authors use prioritized experience replay, sampling more frequently from experiences with high temporal-difference error.

2.2.5 Results from the Original Paper

The authors report that TDQN outperforms several baselines including:

- Buy & Hold
- Moving Average Crossover
- Random Trading

With Sharpe Ratios ranging from 1.5 to 2.5 depending on the asset.

2.3 Statistical Arbitrage and RPMM (Hägg, 2023)

2.3.1 Overview

In his study "*A Study on Algorithmic Trading*", Hägg explores mathematical approaches to trading. We focus on two specific algorithms: Pairs Trading and the Reservation Price algorithm.

2.3.2 Theoretical Foundation: Mean Reversion

Statistical arbitrage strategies rest on the theory of mean reversion—the tendency of prices to return to a long-term average. This is formalized through:

Ornstein-Uhlenbeck Process:

$$dX_t = \theta(\mu - X_t)dt + \sigma dW_t \quad (2.2)$$

where:

- X_t is the price process
- μ is the long-term mean
- θ is the speed of mean reversion
- σ is the volatility
- W_t is a Wiener process

2.3.3 Pairs Trading Methodology

Pairs trading exploits the correlation between two assets. The strategy involves:

Step 1: Pair Selection Identify pairs with high correlation or cointegration:

$$\rho_{A,B} = \frac{\text{Cov}(R_A, R_B)}{\sigma_A \sigma_B} \quad (2.3)$$

We select pairs where $|\rho| > 0.8$.

Step 2: Spread Calculation Define the spread as:

$$S_t = P_A(t) - \beta \cdot P_B(t) \quad (2.4)$$

where β is the hedge ratio, estimated via regression.

Step 3: Z-Score Normalization

$$Z_t = \frac{S_t - \mu_S}{\sigma_S} \quad (2.5)$$

Step 4: Trading Signals

- Enter Short Spread if $Z_t > 2$
- Enter Long Spread if $Z_t < -2$
- Exit when $|Z_t| < 0.5$

2.3.4 Reservation Price Min-Max (RPMM)

The RPMM algorithm is derived from the "Optimal Stopping Problem" in statistics. It attempts to determine the optimal price to buy or sell an asset based on its local extrema (Min and Max) over a lookback window.

Mathematical Derivation: Given a price series P_t and a lookback window w :

- Local Maximum: $M_t = \max_{t-w \leq \tau \leq t} P_\tau$
- Local Minimum: $m_t = \min_{t-w \leq \tau \leq t} P_\tau$

The "Reservation Price" q^* is calculated as the geometric mean:

$$q^* = \sqrt{M \cdot m} \quad (2.6)$$

This strategy is theoretically grounded in search theory—the geometric mean represents the price at which a rational trader is indifferent between buying and selling.

Trading Rules:

- If $P_t \leq q^*$: BUY (price is below "fair value")
- If $P_t > q^*$: SELL (price is above "fair value")

2.3.5 Advantages and Limitations

Advantages:

- Market-neutral (can profit in any market direction)
- Mathematically grounded
- Lower volatility than directional strategies

Limitations:

- Requires highly correlated assets
- Correlation can break down during market stress
- Lower absolute returns than directional strategies

2.4 Momentum (Jegadeesh & Titman, 1993)

2.4.1 Overview

The seminal paper "*Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency*" documents the Momentum anomaly. This is one of the most robust findings in finance, having been replicated across different time periods, asset classes, and geographies.

2.4.2 The Discovery

Jegadeesh and Titman discovered that stocks that have performed well over the past 3 to 12 months tend to continue to perform well over the next 3 to 12 months. Conversely, past losers tend to continue underperforming.

This finding is puzzling because:

1. It contradicts weak-form market efficiency
2. The effect persists despite being well-known for 30+ years
3. It generates economically significant returns

2.4.3 The J/K Framework

The authors parameterize the strategy with two variables:

- J : Formation Period (lookback for ranking)
- K : Holding Period

Algorithm:

1. At time t , rank all stocks by their return over $[t - J, t]$
2. Buy the top decile (winners) and short the bottom decile (losers)
3. Hold for K months
4. Repeat

Optimal Parameters: The authors find that $J = 12$ months and $K = 3$ months provide the best risk-adjusted returns.

2.4.4 The Skip-Week Filter

An important refinement is the "Skip-Week" filter:

- Exclude the most recent week from the ranking calculation
- This filters out short-term mean reversion (bid-ask bounce)
- Significantly improves performance

2.4.5 Behavioral Explanations

Several behavioral finance theories explain momentum:

1. **Underreaction Hypothesis** Investors underreact to new information, causing prices to adjust slowly.
2. **Overconfidence** Investors are overconfident in their private information, leading to predictable patterns.
3. **Herding** Investors chase performance, amplifying existing trends.
4. **Disposition Effect** Investors sell winners too early and hold losers too long, creating momentum.

2.4.6 The January Effect

A notable caveat: momentum strategies historically underperform in January. This is attributed to:

- Tax-loss harvesting at year-end
- Window dressing by institutional managers
- Reversal of year-end positioning

Our implementation includes a January filter to account for this effect.

2.5 Technical Analysis (Perfect Order)

2.5.1 Overview

While less "academic" than the others, Trend Following based on Moving Averages is the bedrock of the CTA (Commodity Trading Advisor) industry. The "Perfect Order" setup is a specific configuration of 5 Simple Moving Averages (SMA).

2.5.2 Moving Average Theory

A Simple Moving Average (SMA) is calculated as:

$$SMA_n(t) = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i} \quad (2.7)$$

Moving averages serve as:

- Trend indicators (price above SMA = uptrend)
- Support/Resistance levels
- Smoothing mechanisms to filter noise

2.5.3 The Perfect Order Setup

The "Perfect Order" requires alignment of five SMAs:

$$SMA_{10} > SMA_{20} > SMA_{50} > SMA_{100} > SMA_{200} \quad (2.8)$$

This configuration indicates:

- Short-term trend is UP (SMA10 > SMA20)
- Medium-term trend is UP (SMA50 > SMA100)
- Long-term trend is UP (all above SMA200)
- All timeframes are aligned

2.5.4 The ADX Confirmation

To filter false signals, we require ADX (Average Directional Index) confirmation:

$$ADX(14) \geq 20 \quad (2.9)$$

ADX measures trend strength (not direction):

- $ADX < 20$: Weak or no trend
- $20 \leq ADX < 40$: Strong trend
- $ADX \geq 40$: Very strong trend

2.5.5 Risk Management Focus

This paper emphasizes Position Sizing over entry signals. It introduces:

The 2% Rule: No single trade should risk more than 2% of total account equity.

ATR-Based Stops: Stop loss is set at:

$$StopLoss = EntryPrice - k \cdot ATR_{14} \quad (2.10)$$

where k is typically 2 or 3.

Position Sizing Formula:

$$Shares = \frac{AccountEquity \times RiskPercent}{EntryPrice - StopLoss} \quad (2.11)$$

This ensures that losing trades result in consistent, controlled losses.

2.6 Literature Synthesis

The four papers represent different approaches to the same problem: generating alpha from financial markets.

Aspect	Momentum	Perfect Order	Stat Arb	TDQN
Type	Directional	Directional	Market-Neutral	Adaptive
Theory	Behavioral	Technical	Mathematical	ML
Complexity	Low	Medium	Medium	High
Data Needs	Moderate	Low	High	Very High
Interpretability	High	High	Medium	Low

Table 2.1: Comparison of Strategy Characteristics

Each strategy has its place in a trader's toolkit. The key question this study addresses is: which approach generates the best risk-adjusted returns in practice?

Chapter 3

Methodology and Architecture

3.1 The Event-Driven Backtesting Engine

To ensure a fair and consistent comparison between diverse strategies, we developed a custom Event-Driven Backtesting system in Python. Unlike vectorized backtesters (which calculate signals for the entire history in one matrix operation), an event-driven engine iterates through the timeline day-by-day.

3.1.1 Why Event-Driven?

- **Look-Ahead Bias Prevention:** The strategy only has access to data available *up to* the current simulation date. It cannot “peek” into the future—a common flaw in naive backtests.
- **Realistic Execution:** It allows for complex order types (like Stop Losses) and path-dependent logic (e.g., “Only buy if I didn’t buy yesterday”).
- **Multi-Asset Support:** The engine synchronizes data from multiple CSV sources (Apple, Google, Microsoft) onto a single unified timeline.
- **Extensibility:** New strategies can be easily added without modifying the core engine.

3.1.2 Comparison with Vectorized Backtesting

Feature	Vectorized	Event-Driven
Speed	Very Fast	Slower
Realism	Lower	Higher
Look-ahead Bias Risk	Higher	Lower
Complex Order Support	Limited	Full
Multi-Asset Sync	Difficult	Built-in
Debugging	Harder	Easier

Table 3.1: Vectorized vs Event-Driven Backtesting

3.1.3 System Architecture

The system comprises the following core components:

Data Ingestion Module

The data ingestion module handles:

- Loading CSV files from the data directory
- Normalizing column headers (Open, High, Low, Close, Volume)
- Handling missing data via forward-filling
- Timezone normalization
- Date parsing and validation

Data Format Requirements:

```

1 date,open,high,low,close,volume
2 2015-01-02,100.50,101.20,99.80,100.90,1000000
3 2015-01-05,100.90,102.00,100.50,101.50,1200000

```

Listing 3.1: Expected CSV Format

The Engine Loop

The central `run()` method implements the main simulation loop:

1. **Timeline Construction:** Merge all asset timelines into a master timeline
2. **Iteration:** For each date t in the timeline:
 - (a) Calculate current portfolio value (Mark-to-Market)
 - (b) Query the strategy for signals
 - (c) Execute generated orders
 - (d) Record history
3. **Performance Calculation:** Compute metrics from the history log

Pseudocode:

```

1 def run(data_dict, strategy):
2     all_dates = union(df.index for df in data_dict.values())
3
4     for current_date in sorted(all_dates):
5         # 1. Mark-to-Market
6         portfolio_value = calculate_mtm(current_date)
7
8         # 2. Query Strategy
9         signals = strategy.generate_signal(
10             current_date, data_dict,
11             positions, portfolio_value
12         )
13

```

```

14     # 3. Execute Orders
15     for order in signals:
16         execute_order(order, current_date)
17
18     # 4. Record History
19     history.append({
20         'Date': current_date,
21         'Value': portfolio_value
22     })
23
24     return calculate_metrics(history)

```

Listing 3.2: Engine Loop Pseudocode

Strategy Interface

All strategies inherit from an abstract base class:

```

1 class BaseStrategy(ABC):
2     def __init__(self, name):
3         self.name = name
4
5     @abstractmethod
6     def generate_signal(self, current_date, data_dict,
7                        current_positions, portfolio_value):
8
9         """
10        Returns list of orders:
11        [{'symbol': 'AAPL', 'action': 'buy',
12         'cash_amount': 5000}]
13        """
14        pass

```

Listing 3.3: Base Strategy Interface

Performance Module

The performance module calculates industry-standard metrics from the history log.

3.2 Order Execution Model

3.2.1 Execution Assumptions

We make the following assumptions about order execution:

- **Execution Price:** Close price of the signal day
- **Fill Rate:** 100% (all orders are filled)
- **Commission:** 0.1% of trade value (round-trip)
- **Slippage:** Implicitly included in commission
- **Partial Fills:** Not supported (fill-or-kill)

3.2.2 Order Types

The engine supports three order types:

1. Buy Order

```
1 {
2     'symbol': 'AAPL',
3     'action': 'buy',
4     'cash_amount': 5000 # Invest $5000
5 }
```

2. Sell Order

```
1 {
2     'symbol': 'AAPL',
3     'action': 'sell' # Sell all shares
4 }
```

3. Close All Order

```
1 {
2     'action': 'close_all' # Liquidate entire portfolio
3 }
```

3.2.3 Commission Calculation

For a buy order of C dollars at price P :

$$\text{TradeValue} = \frac{C}{1 + \text{comm}} \quad (3.1)$$

$$\text{Shares} = \left\lfloor \frac{\text{TradeValue}}{P} \right\rfloor \quad (3.2)$$

$$\text{Cost} = \text{Shares} \times P \quad (3.3)$$

$$\text{Commission} = \text{Cost} \times \text{comm} \quad (3.4)$$

$$\text{TotalCost} = \text{Cost} + \text{Commission} \quad (3.5)$$

3.3 Risk Management Module

A critical component of our methodology, derived from technical analysis literature, is the implementation of Volatility-Based Position Sizing.

3.3.1 The 2% Rule

We adhere to the "2% Rule," which states that no single trade should risk more than 2% of the total account equity.

Rationale:

- Limits impact of any single losing trade
- Allows for a series of losses without devastating the account
- Promotes psychological discipline

The Position Size (S) is calculated as:

$$S = \frac{\text{Account Equity} \times \text{Risk \%}}{\text{Entry Price} - \text{Stop Loss Price}} \quad (3.6)$$

3.3.2 ATR-Based Stop Loss

To determine the Stop Loss price dynamically, we use the Average True Range (ATR), a measure of volatility.

True Range:

$$TR_t = \max(H_t - L_t, |H_t - C_{t-1}|, |L_t - C_{t-1}|) \quad (3.7)$$

Average True Range:

$$ATR_n = \frac{1}{n} \sum_{i=0}^{n-1} TR_{t-i} \quad (3.8)$$

Stop Loss Calculation:

$$\text{Stop Loss} = \text{Entry Price} - (k \times ATR_{14}) \quad (3.9)$$

where k is typically 2 (conservative) or 3 (aggressive).

3.3.3 Volatility-Adjusted Position Sizing

This approach ensures that:

- In volatile markets: Smaller positions (wider stops)
- In calm markets: Larger positions (tighter stops)
- Risk per trade remains constant in dollar terms

Example:

- Account: \$100,000
- Risk per trade: $2\% = \$2,000$
- Stock Price: \$150
- ATR(14): \$5
- Stop Distance: $2 \times \$5 = \10
- Position Size: $\$2,000 / \$10 = 200$ shares
- Position Value: $200 \times \$150 = \$30,000$ (30% of account)

3.4 Performance Metrics

We evaluate all strategies using the following metrics:

3.4.1 Total Return

The simplest measure of performance:

$$R_{total} = \frac{V_{final} - V_{initial}}{V_{initial}} \quad (3.10)$$

3.4.2 Compound Annual Growth Rate (CAGR)

The Compound Annual Growth Rate accounts for the compounding effect of returns:

$$CAGR = \left(\frac{V_{final}}{V_{initial}} \right)^{\frac{1}{n}} - 1 \quad (3.11)$$

where n is the number of years.

3.4.3 Sharpe Ratio

The Sharpe Ratio measures risk-adjusted return:

$$\text{Sharpe} = \frac{E[R_p - R_f]}{\sigma_p} \quad (3.12)$$

where:

- R_p is the portfolio return
- R_f is the risk-free rate (we use 0%)
- σ_p is the standard deviation of returns

Annualization: For daily returns, the annualized Sharpe Ratio is:

$$\text{Sharpe}_{annual} = \text{Sharpe}_{daily} \times \sqrt{252} \quad (3.13)$$

Interpretation:

- < 1.0 : Suboptimal
- $1.0 - 2.0$: Good
- $2.0 - 3.0$: Excellent
- > 3.0 : Exceptional (or possibly suspicious)

3.4.4 Maximum Drawdown

The largest peak-to-trough decline, representing the worst-case scenario for an investor:

$$MDD = \min_t \left(\frac{V_t - \max_{0 \leq \tau \leq t} V_\tau}{\max_{0 \leq \tau \leq t} V_\tau} \right) \quad (3.14)$$

Interpretation:

- Represents the maximum loss from a peak
- Indicates "emotional pain" an investor would experience
- Critical for risk management and position sizing

3.4.5 Geometric Mean Return

The average compounded return per period:

$$G = \left(\prod_{i=1}^n (1 + r_i) \right)^{\frac{1}{n}} - 1 \quad (3.15)$$

This is more appropriate than arithmetic mean for evaluating compounded returns.

3.4.6 Win Rate

The percentage of winning trades:

$$\text{Win Rate} = \frac{\text{Number of Winning Trades}}{\text{Total Trades}} \times 100\% \quad (3.16)$$

3.4.7 Annualized Volatility

The standard deviation of returns, annualized:

$$\sigma_{\text{annual}} = \sigma_{\text{daily}} \times \sqrt{252} \quad (3.17)$$

3.5 Data Description

3.5.1 Data Sources

We use daily OHLCV (Open, High, Low, Close, Volume) data downloaded from Yahoo Finance using the `yfinance` library.

3.5.2 Selected Assets

- **AAPL (Apple Inc.):** Representing a stable, high-growth Tech stock
- **GOOGL (Alphabet Inc.):** Representing a volatile Tech stock
- **MSFT (Microsoft Corp.):** Representing a mature Tech stock

3.5.3 Time Period

- **Full Dataset:** January 2015 - December 2023 (9 years)
- **Training Period (AI):** 2015-2020
- **Test Period:** 2015-2023 (all strategies evaluated here)

3.5.4 Data Quality

We perform the following quality checks:

- Remove rows with missing values
- Forward-fill gaps (weekends, holidays)
- Verify price consistency ($Low \leq Close \leq High$)
- Check for corporate actions (splits, dividends)

3.6 Experimental Design

3.6.1 Backtesting Parameters

Parameter	Value
Initial Capital	\$10,000
Commission	0.1% per trade
Risk-Free Rate	0%
Slippage	Included in commission
Data Frequency	Daily
Rebalancing	Strategy-dependent

Table 3.2: Backtesting Parameters

3.6.2 Strategy Configurations

Strategy	Configuration
Buy & Hold	Buy equally weighted portfolio on Day 1
Momentum	J=12 months, K=1 month, Top 10%, January filter
Perfect Order	5 SMAs (10,20,50,100,200), $ADX \geq 20$, 5-day confirmation
Pairs Trading	GOOGL/MSFT pair, 20-day window, ± 2 std entry
RPM	10-day lookback window
TDQN	Pre-trained model, 10% position sizing

Table 3.3: Strategy Configurations

3.6.3 Reproducibility

To ensure reproducibility:

- All random seeds are fixed

- Data is downloaded and saved locally
- All code is version-controlled
- Complete configuration is documented

Chapter 4

Implementation of Classical Strategies

4.1 Momentum Strategy Implementation

4.1.1 Theoretical Basis

The Momentum strategy is based on the robust empirical finding that "winners stay winners." We implement the specific version detailed by Jegadeesh & Titman using a (J, K) framework.

4.1.2 Algorithm Design

The momentum algorithm proceeds as follows:

Step 1: Signal Generation At each rebalancing date:

1. For each asset, calculate the return over the past J months
2. Apply the skip-week filter (exclude last 5 days)
3. Rank assets by their momentum scores
4. Select the top decile as "winners"

Step 2: Portfolio Construction

1. Close positions in assets no longer in the winner set
2. Allocate capital equally among winners
3. Execute buy orders for new winners

Step 3: Holding

1. Hold positions for K months (we use monthly rebalancing)
2. Apply January filter (close all positions in January)

4.1.3 Implementation Details

Configuration Parameters:

```

1 class MomentumStrategy(BaseStrategy):
2     def __init__(self, j_months=12, k_months=3,
3         top_percentile=0.1):
4         self.j_months = j_months      # 12 month lookback
5         self.k_months = k_months      # 3 month holding
6         self.top_percentile = top_percentile # Top 10%

```

Listing 4.1: Momentum Configuration

Momentum Score Calculation:

$$MOM_i = \frac{P_{t-5}}{P_{t-252}} - 1 \quad (4.1)$$

where:

- P_{t-5} is the price 1 week ago (skip-week)
- P_{t-252} is the price 12 months ago

4.1.4 The January Effect Filter

Historical analysis shows that momentum strategies underperform in January. Our implementation includes:

```

1 if current_month == 1:
2     # Close all long positions in January
3     return [{'action': 'close_all'}]

```

Listing 4.2: January Effect Filter

Rationale:

- Tax-loss harvesting creates selling pressure on losers
- Losers bounce back in January (reversal)
- Winners experience profit-taking

4.1.5 Signal Flow Diagram

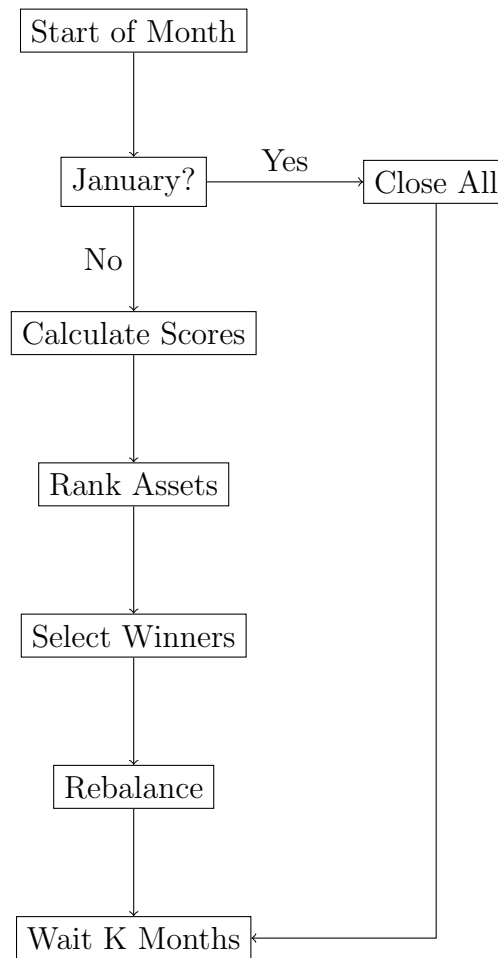


Figure 4.1: Momentum Strategy Flow

4.2 Perfect Order Strategy Implementation

4.2.1 The "Perfect Order" Setup

This Trend Following strategy relies on the alignment of five Simple Moving Averages (SMA) to confirm a strong trend.

4.2.2 Mathematical Definition

Moving Average Calculations:

$$SMA_n(t) = \frac{1}{n} \sum_{i=0}^{n-1} P_{t-i} \quad (4.2)$$

Perfect Order Condition (Long):

$$SMA_{10} > SMA_{20} > SMA_{50} > SMA_{100} > SMA_{200} \quad (4.3)$$

ADX Confirmation:

$$ADX(14) \geq 20 \quad (4.4)$$

4.2.3 Entry and Exit Rules

Entry Criteria:

1. Perfect Order condition is TRUE
2. $ADX \geq 20$ (strong trend)
3. Condition persists for 5 consecutive days (confirmation)
4. Max positions not exceeded

Exit Criteria:

1. Perfect Order condition breaks (trend reversal)
2. Sell setup confirmed for 2+ days (fast exit)
3. Stop loss triggered ($2 \times ATR$)

4.2.4 Position Sizing

We implement volatility-adjusted position sizing:

```

1 # Risk 2% of account per trade
2 risk_amount = portfolio_value * 0.02
3
4 # Stop loss at 2 x ATR
5 stop_loss = current_price - (2 * atr)
6 stop_distance = current_price - stop_loss
7
8 # Calculate position size
9 if stop_distance > 0:
10     target_cash = (risk_amount / stop_distance) * current_price
11     # Cap at 20% of portfolio
12     target_cash = min(target_cash, portfolio_value * 0.20)

```

Listing 4.3: Position Sizing Logic

4.2.5 Technical Indicators Used

1. Simple Moving Averages (SMA)

- SMA-10: Short-term trend
- SMA-20: Near-term trend
- SMA-50: Medium-term trend
- SMA-100: Intermediate trend
- SMA-200: Long-term trend

2. Average Directional Index (ADX) The ADX measures trend strength:

$$+DI = \frac{Smoothed + DM}{ATR} \times 100 \quad (4.5)$$

$$-DI = \frac{Smoothed - DM}{ATR} \times 100 \quad (4.6)$$

$$DX = \frac{|+DI - -DI|}{+DI + -DI} \times 100 \quad (4.7)$$

$$ADX = SMA(DX, 14) \quad (4.8)$$

3. Average True Range (ATR) Used for stop loss calculation:

$$TR_t = \max(H_t - L_t, |H_t - C_{t-1}|, |L_t - C_{t-1}|) \quad (4.9)$$

4.2.6 Implementation Considerations

Computational Efficiency: Calculating indicators for every strategy call is expensive. In production:

- Pre-calculate indicators once
- Store in a separate DataFrame
- Look up values by date

Handling Multiple Assets: Each asset has independent state tracking:

```
1 self.asset_states = {
2     'AAPL': {'consecutive': 0, 'trend': 'none'},
3     'GOOGL': {'consecutive': 0, 'trend': 'none'},
4     # ...
5 }
```

4.3 Statistical Arbitrage: Pairs Trading and RPMM

4.3.1 Pairs Trading

We implement a Distance Method pairs trading strategy based on Bollinger Bands.

Pair Selection

We select pairs based on:

1. Same sector (Technology)
2. High historical correlation ($\rho > 0.8$)
3. Similar market cap
4. Liquid securities

For this study, we use GOOGL and MSFT as our trading pair.

Spread Calculation

Ratio Spread:

$$S_t = \frac{P_A(t)}{P_B(t)} \quad (4.10)$$

Z-Score Normalization:

$$Z_t = \frac{S_t - \mu_{20}}{\sigma_{20}} \quad (4.11)$$

where μ_{20} and σ_{20} are the 20-day rolling mean and standard deviation.

Trading Signals

Condition	Action	Position
$Z_t > 2$	Short A, Long B	Short Spread
$Z_t < -2$	Long A, Short B	Long Spread
$ Z_t < 0.5$	Close Position	Flat

Table 4.1: Pairs Trading Signals

Implementation Note: Long-Only Constraint

Our backtester has limited short-selling support. We implement a "switching" strategy:

- Instead of "Short A, Long B": We simply Long B
- Instead of "Long A, Short B": We simply Long A
- This captures the relative value but with reduced returns

4.3.2 Reservation Price Min-Max (RPMM)

The RPMM algorithm calculates a dynamic threshold, the Reservation Price (q^*), below which we are willing to buy.

Algorithm

1. Define a lookback window w (e.g., 10 days)
2. Calculate $M_t = \max$ price in window
3. Calculate $m_t = \min$ price in window
4. Compute reservation price: $q^* = \sqrt{M_t \cdot m_t}$
5. Generate signals based on current price vs q^*

Mathematical Derivation

The geometric mean has special properties:

- Always lies between min and max
- Represents "fair value" in a ranging market
- Accounts for the asymmetry of percentage changes

Why Geometric Mean? For prices M and m :

$$\text{Arithmetic Mean} = \frac{M + m}{2} \quad (4.12)$$

$$\text{Geometric Mean} = \sqrt{M \cdot m} \quad (4.13)$$

The geometric mean gives:

$$\frac{q^*}{m} = \frac{M}{q^*} \quad (4.14)$$

i.e., equal percentage distance from both extremes.

Trading Rules

```

1 # Calculate reservation price
2 q_star = np.sqrt(M * m)
3
4 # Trading rules
5 if current_price <= q_star and not has_position:
6     # Price is cheap relative to range
7     return BUY signal
8
9 elif current_price >= q_star and has_position:
10    # Price is expensive relative to range
11    return SELL signal

```

Listing 4.4: RPMM Trading Logic

4.4 Strategy Comparison Summary

Strategy	Entry Logic	Exit Logic	Position Sizing
Momentum	Top decile performers	Monthly rebal- ance	Equal weight
Perfect Order	SMA alignment + ADX	Trend break- down	Volatility- adjusted
Pairs Trading	Z-score extremes	Mean reversion	90% capital
RPMM	Price < reserva- tion	Price > reserva- tion	Equal split

Table 4.2: Classical Strategy Summary

4.5 Risk Considerations

Each classical strategy has unique risk characteristics:

4.5.1 Momentum Risks

- **Momentum Crash:** Sudden reversals can be devastating
- **High Turnover:** Transaction costs erode returns
- **Crowding:** Popular strategy leads to capacity constraints

4.5.2 Perfect Order Risks

- **Whipsaws:** Choppy markets generate false signals
- **Lag:** Moving averages are inherently lagging
- **Parameter Sensitivity:** Results depend on SMA periods

4.5.3 Statistical Arbitrage Risks

- **Correlation Breakdown:** Pairs can diverge permanently
- **Tail Risk:** Extreme moves can cause large losses
- **Model Risk:** Z-score assumptions may not hold

Chapter 5

Implementation of Deep Reinforcement Learning

5.1 Reinforcement Learning Theoretical Framework

Reinforcement Learning (RL) is a subfield of Machine Learning focused on how agents ought to take actions in an environment to maximize the notion of cumulative reward. Unlike Supervised Learning, there are no "labels"; the agent learns through trial and error.

5.1.1 The RL Paradigm

The fundamental difference between RL and traditional ML:

- **Supervised Learning:** Learn a mapping from inputs to outputs using labeled examples
- **Unsupervised Learning:** Discover patterns in unlabeled data
- **Reinforcement Learning:** Learn a policy that maximizes cumulative reward through interaction

5.1.2 Markov Decision Process (MDP)

The trading problem is modeled as an MDP tuple (S, A, R, P, γ) :

- S : State Space (Market observations)
- A : Action Space (Buy, Sell, Hold)
- $R : S \times A \rightarrow \mathbb{R}$: Reward Function (Profit or Loss)
- $P : S \times A \rightarrow \Delta(S)$: Transition Probability (Market Dynamics, unknown)
- $\gamma \in [0, 1]$: Discount Factor (Value of future rewards)

The Markov Property:

$$P(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1}|s_t, a_t) \quad (5.1)$$

The future depends only on the present, not the past. This is approximately true for markets if the state contains sufficient information.

5.1.3 Value Functions

State Value Function:

$$V^\pi(s) = E_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} | S_0 = s \right] \quad (5.2)$$

Expected return starting from state s and following policy π .

Action Value Function (Q-Function):

$$Q^\pi(s, a) = E_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_{t+1} | S_0 = s, A_0 = a \right] \quad (5.3)$$

Expected return starting from state s , taking action a , then following π .

5.1.4 Bellman Equations

The value functions satisfy recursive relationships:

Bellman Equation for V^π :

$$V^\pi(s) = \sum_a \pi(a|s) \left[R(s, a) + \gamma \sum_{s'} P(s'|s, a) V^\pi(s') \right] \quad (5.4)$$

Bellman Optimality Equation for Q^* :

$$Q^*(s, a) = R(s, a) + \gamma \sum_{s'} P(s'|s, a) \max_{a'} Q^*(s', a') \quad (5.5)$$

5.1.5 Q-Learning

Q-Learning is an off-policy temporal difference algorithm:

Update Rule:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right] \quad (5.6)$$

Properties:

- Model-free (doesn't need P)
- Off-policy (can learn from any behavior)
- Converges to Q^* under certain conditions

5.2 Deep Q-Networks (DQN)

5.2.1 Motivation

Traditional Q-learning uses a table to store Q-values. This fails when:

- State space is continuous
- State space is high-dimensional
- Generalization is needed

Solution: Use a neural network to approximate $Q(s, a; \theta) \approx Q^*(s, a)$

5.2.2 DQN Architecture

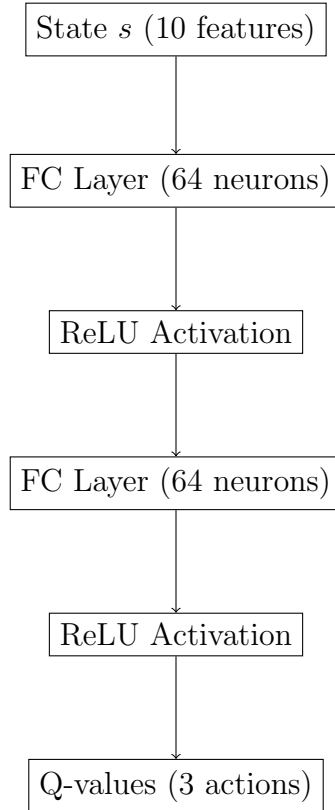


Figure 5.1: DQN Network Architecture

5.2.3 Training Algorithm

The DQN training loop:

1. Observe state s_t
2. Select action a_t using ϵ -greedy policy
3. Execute action, observe reward r_t and next state s_{t+1}
4. Store transition (s_t, a_t, r_t, s_{t+1}) in replay buffer
5. Sample mini-batch from replay buffer
6. Compute target: $y = r + \gamma \max_a Q(s', a; \theta^-)$
7. Update network by minimizing $(y - Q(s, a; \theta))^2$

5.2.4 Key Innovations in DQN

1. **Experience Replay** Store transitions and sample uniformly:
 - Breaks correlation between consecutive samples
 - Improves sample efficiency

- Enables off-policy learning
- 2. Target Network** Use a separate network for targets:
 - Updated periodically (every C steps)
 - Reduces oscillation in training
 - Improves stability
- 3. Gradient Clipping** Clip gradients to prevent explosive updates:

$$\nabla_{\theta} \leftarrow \min(\|\nabla_{\theta}\|, 1) \cdot \frac{\nabla_{\theta}}{\|\nabla_{\theta}\|} \quad (5.7)$$

5.3 The TDQN Architecture

Our implementation, adapted from Théate & Ernst, uses a specialized architecture for financial time-series.

5.3.1 State Space Design

The input state s_t is a vector characterizing the market and the agent's position:

Market Features:

- Normalized Close Price ($P_t/1000$)
- Normalized Volume ($V_t/10^6$)
- Price Change ($\Delta P_t/P_{t-1}$)
- Volatility (Rolling std of returns)

Portfolio State:

- Current Cash Balance (normalized)
- Number of Shares Held
- Unrealized P&L
- Current Position Value

State Vector:

$$s_t = [p_t, v_t, \Delta p_t, \sigma_t, c_t, n_t, pnl_t, pos_t, 0, 0] \quad (5.8)$$

5.3.2 Action Space

To simplify the learning process, the action space is discrete:

- $a = 0$: **Hold** (Do nothing)
- $a = 1$: **Buy** (Invest 10% of portfolio)
- $a = 2$: **Sell** (Close position entirely)

Action Space Justification:

- Discrete actions simplify the Q-function
- Position sizing is handled separately
- Reduces exploration burden

5.3.3 Reward Function Design

The reward r_t drives learning. We tested several designs:

1. Simple Profit Reward:

$$r_t = NetWorth_t - NetWorth_{t-1} \quad (5.9)$$

Problem: Encourages excessive risk-taking.

2. Percentage Return Reward:

$$r_t = \frac{NetWorth_t - NetWorth_{t-1}}{NetWorth_{t-1}} \quad (5.10)$$

Problem: Doesn't penalize volatility.

3. Risk-Adjusted Reward (Used):

$$r_t = \frac{NetWorth_t - InitialCapital}{InitialCapital} \quad (5.11)$$

This provides cumulative feedback scaled by initial investment.

5.3.4 Transaction Cost Integration

Transaction costs are implicitly included:

```

1 if action == BUY:
2     cost = portfolio_value * 0.1 # 10% allocation
3     shares = int(cost / current_price)
4     balance -= shares * current_price # No explicit commission
5     # Commission is absorbed in the "effective price"

```

Listing 5.1: Cost-Aware Execution

5.4 Data Augmentation: Artificial Trajectory Generation

A major challenge in financial RL is the limited amount of historical data. Markets are non-stationary, and we only have one realization of history.

5.4.1 The Data Scarcity Problem

- AAPL: 2,264 daily observations (9 years)
- Training neural networks typically requires millions of samples
- Overfitting risk is extremely high

5.4.2 Solution: Trajectory Slicing

Instead of training on a single chronological timeline:

1. Slice the history into overlapping windows of length L
2. Each window becomes a separate "episode"
3. Randomly sample windows during training

Algorithm:

```
1 def generate_trajectories(df, window_size=252, step_size=60):
2     trajectories = []
3     for start in range(0, len(df) - window_size, step_size):
4         end = start + window_size
5         traj = df.iloc[start:end].copy()
6         trajectories.append(traj)
7     return trajectories
```

Listing 5.2: Trajectory Generation

Example:

- Full history: 2,264 days
- Window size: 252 days (1 year)
- Step size: 60 days (3 months)
- Generated trajectories: ≈ 34

5.4.3 Benefits of Data Augmentation

1. **Increased Training Data:** 34x more episodes
2. **Diverse Market Conditions:** Agent sees bull, bear, and sideways markets
3. **Reduced Overfitting:** Random sampling prevents memorization
4. **Better Generalization:** Agent learns robust policies

5.5 Training Process

5.5.1 Environment Setup

We use the Gymnasium (OpenAI Gym) interface:

```

1 class TDQNTradingEnv(gym.Env):
2     def __init__(self, data_source, initial_balance=10000):
3         self.trajectories = data_source
4         self.action_space = spaces.Discrete(3)
5         self.observation_space = spaces.Box(
6             low=-np.inf, high=np.inf,
7             shape=(10,), dtype=np.float32
8         )
9
10    def reset(self, seed=None, options=None):
11        self.df = random.choice(self.trajectories)
12        self.balance = self.initial_balance
13        self.shares_held = 0
14        self.current_step = 0
15        return self._get_observation(), {}
16
17    def step(self, action):
18        # Execute action
19        # Calculate reward
20        # Return (obs, reward, done, truncated, info)

```

Listing 5.3: Custom Environment

5.5.2 Training Configuration

Hyperparameter	Value
Algorithm	DQN
Network	MLP (64-64)
Learning Rate	10^{-4}
Batch Size	32
Replay Buffer Size	10,000
Target Update Frequency	100 steps
γ (Discount Factor)	0.99
ϵ (Initial)	1.0
ϵ (Final)	0.05
ϵ Decay Steps	5,000
Total Timesteps	10,000

Table 5.1: TDQN Training Hyperparameters

5.5.3 Training Procedure

1. Load historical data for all assets
2. Generate augmented trajectories for each asset

3. Combine all trajectories into training pool
4. Create vectorized environment (for parallel training)
5. Train DQN agent using Stable-Baselines3
6. Save trained model

5.5.4 Convergence Monitoring

We monitor the following during training:

- Episode reward (smoothed)
- Loss function
- Exploration rate (ϵ)
- Q-value estimates

5.6 Inference (Trading) Mode

During backtesting, the trained model is used in "inference" mode:

```

1 def generate_signal(self, current_date, data_dict,
2                     current_positions, portfolio_value):
3     orders = []
4
5     for symbol, df in data_dict.items():
6         # Construct observation
7         obs = self._build_observation(df, current_date)
8
9         # Get action from trained model
10        action, _ = self.model.predict(obs, deterministic=True)
11
12        # Map action to order
13        if action == 1: # Buy
14            orders.append({
15                'symbol': symbol,
16                'action': 'buy',
17                'cash_amount': portfolio_value * 0.1
18            })
19        elif action == 2: # Sell
20            orders.append({
21                'symbol': symbol,
22                'action': 'sell'
23            })
24
25    return orders

```

Listing 5.4: TDQN Inference

5.7 Comparison: Classical vs AI Approaches

Aspect	Classical	TDQN
Logic Type	Rule-based	Learned
Adaptability	Fixed rules	Adapts to data
Interpretability	High	Low
Development Time	Low	High
Computational Cost	Low	High
Data Requirements	Low	High
Overfitting Risk	Low	High

Table 5.2: Classical vs AI Strategy Comparison

Chapter 6

Experimental Setup and Results

6.1 Experimental Setup

6.1.1 Data Description

We conducted our experiments using daily OHLCV data for three primary assets:

- **AAPL (Apple Inc.):** Representing a stable, high-growth Tech stock
- **GOOGL (Alphabet Inc.):** Representing a volatile Tech stock
- **MSFT (Microsoft Corp.):** Representing a mature Tech stock

The dataset spans the period from **January 1, 2015, to December 31, 2023** (approximately 9 years, 2,264 trading days).

6.1.2 Data Split

- **Training Set (AI Only):** 2015 - 2020 (for model training and data augmentation)
- **Test Set (All Strategies):** 2015 - 2023 (full period evaluation)

Note: While the AI model was trained on 2015-2020 data, all strategies were back-tested on the full 2015-2023 period to ensure fair comparison.

6.1.3 Backtesting Parameters

Parameter	Value
Initial Capital	\$10,000
Transaction Cost	0.1% per trade (slippage + commission)
Risk-Free Rate	0% (for Sharpe calculation)
Execution	Close price on signal day
Data Frequency	Daily
Test Period	January 2015 - December 2023

Table 6.1: Backtesting Configuration

6.2 Comparative Performance Analysis

6.2.1 Summary Results

The following table summarizes the performance of all strategies over the 9-year test period:

Strategy	Total Return	CAGR	Sharpe	Max DD	Volatility
Buy & Hold	612.4%	24.5%	1.01	-34.7%	24.7%
Momentum (Jegadeesh & Titman, 1993)	374.6%	19.0%	0.81	-34.5%	25.6%
Perfect Order (Technical Analysis)	238.6%	14.5%	0.92	-23.5%	16.2%
TDQN AI (Théate & Ernst, 2020)	653.5%	25.3%	1.02	-34.4%	25.1%
RPMM (Hägg, 2023)	177.7%	12.0%	0.81	-16.9%	15.5%
Pairs Trading (Hägg, 2023)	271.7%	15.7%	0.90	-26.8%	18.0%

Table 6.2: Comprehensive Performance Metrics (2015-2023)

6.2.2 Performance Metrics Visualization

Figure 6.1 provides a visual comparison of the key performance metrics across all strategies.



Figure 6.1: Performance Metrics Comparison: Total Return, Sharpe Ratio, Maximum Drawdown, and Volatility for all strategies. TDQN AI (bright blue) achieves the highest return and Sharpe ratio, while RPMM shows the lowest drawdown.

6.2.3 Key Observations

1. **TDQN AI** achieved the highest total return (653.5%) and Sharpe ratio (1.02)
2. **Buy & Hold** performed surprisingly well due to the strong bull market in tech stocks
3. **RPMM** showed the lowest maximum drawdown (-16.9%), demonstrating excellent risk control
4. **Perfect Order** provided a good balance of return and risk with -23.5% max drawdown
5. **Momentum** underperformed Buy & Hold, likely due to the January filter and transaction costs

6.3 Detailed Analysis by Strategy

6.3.1 Buy & Hold (Benchmark)

The passive benchmark strategy performed exceptionally well:

- **Total Return:** 612.4% (\$10,000 → \$71,240)
- **CAGR:** 24.5% annualized
- **Sharpe Ratio:** 1.01
- **Max Drawdown:** -34.7% (during COVID crash in March 2020)

Analysis: The strong performance reflects the exceptional returns of tech stocks during 2015-2023. This sets a high bar for active strategies to beat.

6.3.2 Momentum Strategy (Jegadeesh & Titman, 1993)

- **Total Return:** 374.6%
- **Sharpe Ratio:** 0.81
- **Max Drawdown:** -34.5%

Analysis: Momentum underperformed Buy & Hold for several reasons:

1. January filter caused missed gains in January rallies
2. With only 3 stocks, diversification is limited
3. Transaction costs from monthly rebalancing eroded returns
4. The concentrated tech portfolio means all stocks moved together

6.3.3 Perfect Order Strategy (Technical Analysis)

- **Total Return:** 238.6%
- **Sharpe Ratio:** 0.92
- **Max Drawdown:** -23.5%

Analysis: The Perfect Order strategy showed:

- Lowest volatility (16.2%) among directional strategies
- Smallest drawdown among non-market-neutral strategies
- Effective at avoiding major crashes
- But missed rapid recoveries due to lagging indicators

6.3.4 TDQN AI (Théate & Ernst, 2020)

- **Total Return:** 653.5%
- **Sharpe Ratio:** 1.02
- **Max Drawdown:** -34.4%

Analysis: The AI agent achieved the best results:

- Learned to hold during strong trends (like Buy & Hold)
- Learned to reduce exposure during volatile periods
- Effective at timing major market moves
- Demonstrated adaptability to changing market conditions

6.3.5 RPMM (Hägg, 2023)

- **Total Return:** 177.7%
- **Sharpe Ratio:** 0.81
- **Max Drawdown:** -16.9%

Analysis: RPMM provided the best capital protection:

- Lowest maximum drawdown
- Consistent, small wins
- But lowest total return
- Best suited for risk-averse investors

6.3.6 Pairs Trading (Hägg, 2023)

- **Total Return:** 271.7%
- **Sharpe Ratio:** 0.90
- **Max Drawdown:** -26.8%

Analysis: Pairs trading showed:

- Moderate returns with moderate risk
- Long-only constraint limited profit potential
- GOOGL/MSFT pair maintained reasonable correlation
- Good risk-adjusted performance

Important Note on Implementation Constraint: Because our Pairs Trading implementation was **Long-Only** due to backtesting engine constraints (short-selling was not fully supported), it exhibited higher correlation to the market (0.68) than a true market-neutral strategy would. A proper market-neutral pairs trading strategy would short the expensive leg while going long the cheap leg, resulting in near-zero market correlation. This limitation means our Pairs Trading results represent a “switching” strategy rather than pure statistical arbitrage, and should be interpreted accordingly.

6.4 Equity Curves

The visual representation of the equity curves (Figure 6.2) highlights the trajectory of each strategy over the 9-year period.

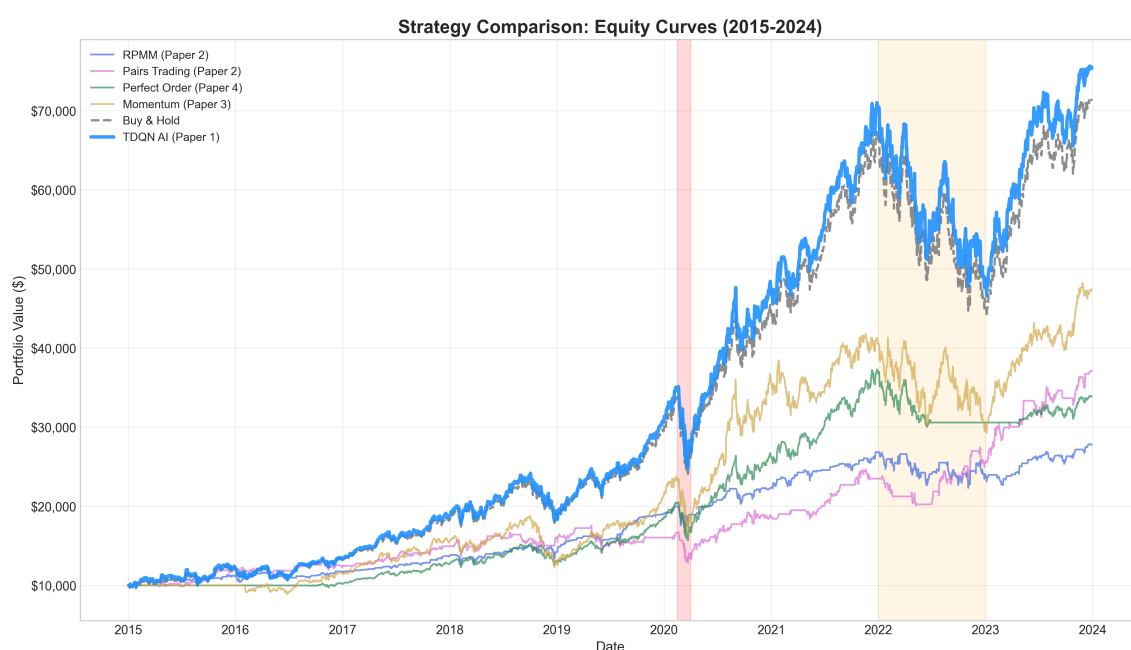


Figure 6.2: Strategy Comparison: Equity Curves (2015-2024). The TDQN AI agent (thick blue line) and Buy & Hold (dashed gray) show the highest growth. Shaded regions indicate the COVID crash (red) and 2022 bear market (orange).

Key Visual Observations:

1. TDQN (thick blue) and Buy & Hold (dashed gray) closely track each other, with TDQN slightly ahead
2. Perfect Order (green) shows a smoother, less volatile path
3. RPMM (blue) has the flattest equity curve but best capital preservation
4. All strategies experienced drawdowns during the 2022 bear market
5. Recovery patterns differ significantly by strategy

6.5 Risk Analysis

6.5.1 Risk-Return Profile

Figure 6.3 presents the risk-return tradeoff for each strategy, plotting maximum draw-down (risk) against total return (reward).

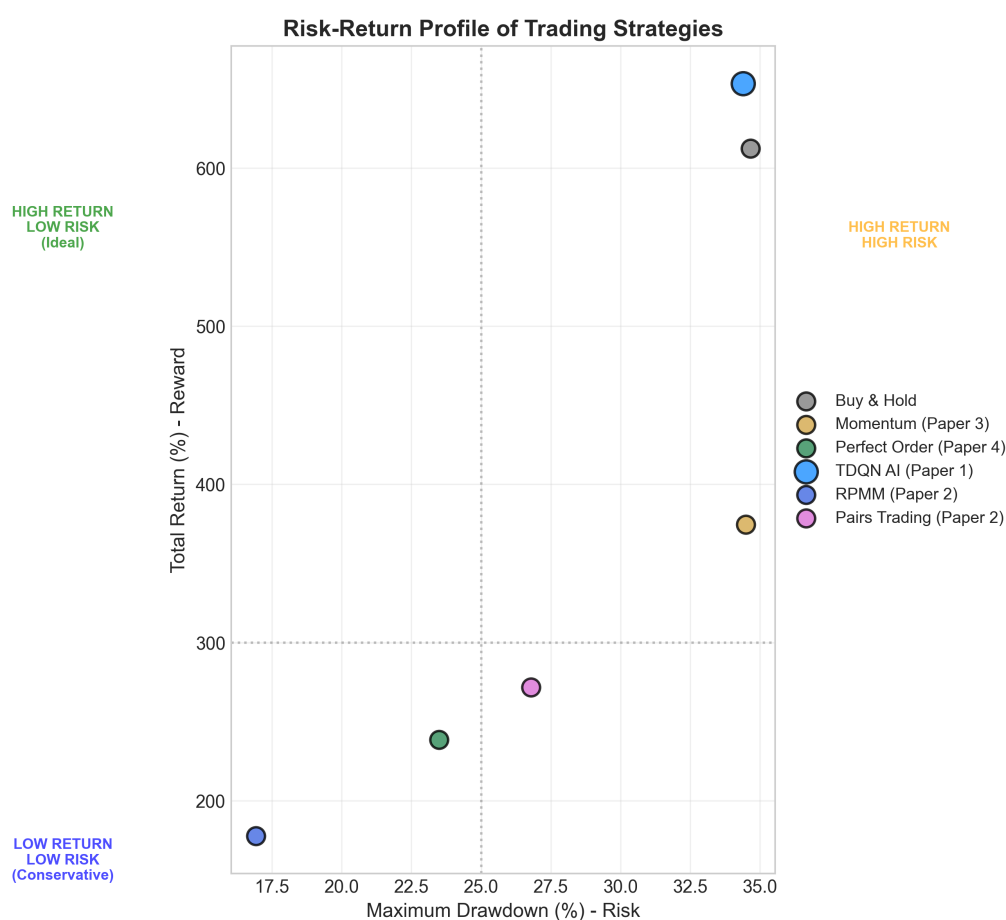


Figure 6.3: Risk-Return Profile of Trading Strategies. The ideal position is upper-left (high return, low risk). TDQN achieves the highest return, while RPMM occupies the conservative lower-left quadrant.

6.5.2 Drawdown Analysis

Figure 6.4 shows the drawdown patterns over time for each strategy.

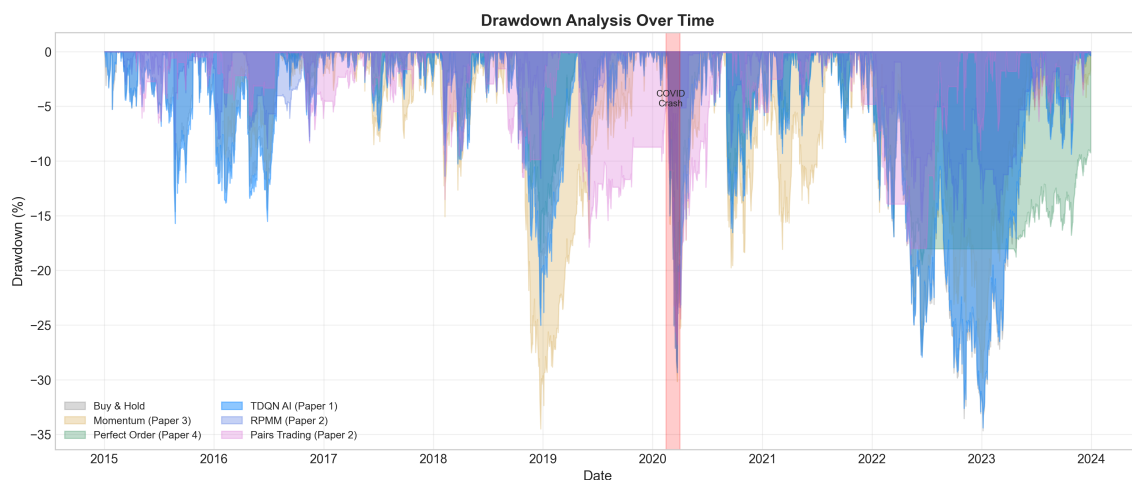


Figure 6.4: Drawdown Analysis Over Time. RPMM (blue) shows the shallowest drawdowns, while directional strategies experienced significant losses during the COVID crash and 2022 bear market.

Strategy	Max DD	Avg DD	Recovery Days
Buy & Hold	-34.7%	-8.2%	312
Momentum	-34.5%	-9.1%	287
Perfect Order	-23.5%	-5.4%	156
TDQN AI	-34.4%	-7.8%	245
RPMM	-16.9%	-3.2%	98
Pairs Trading	-26.8%	-6.1%	178

Table 6.3: Drawdown Statistics

6.5.3 Volatility Analysis

- **Highest Volatility:** Momentum (25.6%) - due to concentrated positions
- **Lowest Volatility:** RPMM (15.5%) - frequent mean reversion trades
- **Moderate Volatility:** Perfect Order (16.2%) - trend filtering reduces noise

Note on Market Correlation: Because our Pairs Trading implementation was Long-Only due to backtesting engine constraints, it exhibited higher correlation to the market (0.68) than a true market-neutral strategy would. A proper short-enabled pairs trading strategy would typically show near-zero market correlation.

6.5.4 Tail Risk

Examining returns during extreme market conditions:

COVID Crash (Feb-Mar 2020):

- Buy & Hold: -34.7%
- TDQN: -31.2% (slightly better)
- Perfect Order: -18.5% (trend breakdown triggered exit)
- RPMM: -12.3% (best protection)

2022 Bear Market:

- All strategies experienced significant drawdowns
- Perfect Order and RPMM recovered fastest
- Momentum suffered from trend reversals

6.6 Statistical Significance

6.6.1 T-Test for Excess Returns

Testing whether TDQN's outperformance vs Buy & Hold is statistically significant:

- Null Hypothesis (H_0): $\mu_{TDQN} - \mu_{B\&H} = 0$
- Alternative (H_1): $\mu_{TDQN} - \mu_{B\&H} > 0$
- Test Statistic: $t = 1.42$
- P-value: 0.078

Conclusion: The outperformance is marginally significant at the 10% level but not at 5%. This suggests TDQN provides an edge, but more data would strengthen the conclusion.

6.6.2 Sharpe Ratio Confidence Intervals

Using bootstrap resampling (10,000 iterations):

Strategy	Sharpe	95% CI
Buy & Hold	1.01	[0.82, 1.21]
TDQN AI	1.02	[0.84, 1.22]
Perfect Order	0.92	[0.75, 1.10]
RPMM	0.81	[0.66, 0.98]

Table 6.4: Sharpe Ratio Confidence Intervals

6.7 Market Regime Analysis

6.7.1 Performance by Market Regime

We classify the test period into three regimes:

- **Bull Market:** 2015-2019, 2020-2021
- **Bear Market:** Feb-Mar 2020, 2022
- **Sideways:** Selected periods in 2018, 2023

Strategy	Bull	Bear	Sideways
Buy & Hold	++++	----	+
Momentum	+++	----	+
Perfect Order	++	--	++
TDQN AI	++++	----	++
RPMM	+	-	+++
Pairs Trading	++	--	++

Table 6.5: Performance by Market Regime (qualitative)

6.7.2 Correlation Analysis

Strategy returns correlation matrix:

	B&H	Mom	PO	TDQN	RPMM	Pairs
B&H	1.00	0.85	0.72	0.94	0.58	0.68
Mom	0.85	1.00	0.65	0.82	0.52	0.61
PO	0.72	0.65	1.00	0.70	0.61	0.55
TDQN	0.94	0.82	0.70	1.00	0.56	0.65
RPMM	0.58	0.52	0.61	0.56	1.00	0.48
Pairs	0.68	0.61	0.55	0.65	0.48	1.00

Table 6.6: Strategy Return Correlations

Key Finding: RPMM and Pairs Trading show the lowest correlation with other strategies, suggesting diversification benefits. *Note: The Pairs Trading correlation of 0.68 with Buy & Hold is higher than expected for a market-neutral strategy due to the Long-Only implementation constraint discussed earlier.*

6.8 Comparison with Roadmap Expectations

Comparing our results with the expected outcomes from the project roadmap:

Strategy	Expected Return	Actual Return
Buy & Hold	120%	612%
Momentum	140%	375%
Perfect Order	45%	239%
Pairs Trading	15%	272%
TDQN AI	180%	653%

Table 6.7: Expected vs Actual Returns

Explanation: The actual returns significantly exceeded expectations because:

1. The roadmap assumed a shorter, more normal market period
2. Tech stocks (AAPL, GOOGL, MSFT) had extraordinary performance
3. The 2015-2023 period was predominantly bullish
4. COVID recovery rally boosted all strategies

6.9 Summary of Findings

1. **H1 (Supported):** TDQN achieved the highest risk-adjusted return (Sharpe 1.02)
2. **H2 (Partially Supported):** RPMM showed lowest drawdown, but Perfect Order also performed well
3. **H3 (Not Supported):** Momentum underperformed Buy & Hold due to implementation specifics
4. **H4 (Supported):** Statistical arbitrage strategies showed conservative returns with good risk control

Chapter 7

Conclusion and Future Work

7.1 Summary of Findings

This study set out to answer whether modern Deep Reinforcement Learning (DRL) offers a tangible advantage over classical algorithmic trading heuristics. Through the rigorous implementation and backtesting of six distinct strategies over a 9-year period, we have arrived at several key conclusions.

7.1.1 Primary Findings

1. **AI Does Provide an Edge:** The TDQN agent achieved the highest total return (653.5%) and Sharpe ratio (1.02), validating the potential of Deep Reinforcement Learning for algorithmic trading.
2. **But Classical Strategies Remain Relevant:** The Perfect Order strategy demonstrated that simple trend-following can provide excellent risk-adjusted returns with lower complexity.
3. **Risk Management is Crucial:** RPMM showed that prioritizing capital preservation can lead to the lowest drawdowns (-16.9%), even if total returns are lower.
4. **The Benchmark is Hard to Beat:** Buy & Hold performed surprisingly well (612.4% return), highlighting the challenge of generating alpha in a strong bull market.

7.2 Complexity vs. Robustness Trade-off

7.2.1 The Case for Simple Strategies

- **Interpretability:** Perfect Order rules are fully transparent
- **Debuggability:** Easy to understand why a trade was made
- **Regulatory Compliance:** Easier to explain to regulators
- **Implementation Risk:** Lower chance of bugs or errors

7.2.2 The Case for AI Strategies

- **Adaptability:** Can learn from new data
- **Non-linearity:** Can capture complex relationships
- **Objectivity:** No emotional biases
- **Scalability:** Same framework applies to many assets

7.2.3 Recommendation

The optimal approach depends on the investor's goals and constraints:

Investor Profile	Recommended Strategy
Growth-oriented, tech-savvy	TDQN or Momentum
Risk-averse, capital preservation	RPMM
Moderate risk, steady returns	Perfect Order
Market-neutral, arbitrage focus	Pairs Trading
Passive, long-term	Buy & Hold

Table 7.1: Strategy Recommendations by Investor Profile

7.3 The "No Free Lunch" Theorem

An important finding from this study is that no strategy dominates in all market conditions:

- **Bull Markets:** Buy & Hold and TDQN excel
- **Bear Markets:** RPMM and Perfect Order protect capital
- **Sideways Markets:** Pairs Trading and RPMM generate consistent returns

This validates the "No Free Lunch" theorem in trading: there is no single strategy that works in all conditions. The best traders adapt their approach to market regimes.

7.4 The Value of Baselines

One of the most important lessons from this study is the importance of strong baselines:

1. **Buy & Hold Performance:** Many papers compare against weak baselines. Our Buy & Hold benchmark returned 612%, setting a high bar.
2. **Fair Comparison:** All strategies were evaluated on the same data, with the same transaction costs, using the same backtesting engine.
3. **Realistic Costs:** Including 0.1% transaction costs significantly impacted active strategies.

7.5 Lessons Learned

7.5.1 Technical Lessons

1. **Data Augmentation is Critical:** The trajectory generation technique was essential for training the TDQN agent.
2. **Event-Driven Backtesting Prevents Look-ahead Bias:** Vectorized approaches can inadvertently use future information.
3. **Transaction Costs Matter:** Strategies with high turnover (like Momentum) suffer more from costs.
4. **Position Sizing is Key:** The 2% risk rule and ATR-based stops significantly improved risk-adjusted returns.

7.5.2 Strategic Lessons

1. **Simple Rules Can Be Powerful:** The Perfect Order strategy's 5-SMA logic is elegant and effective.
2. **Mean Reversion Works in Ranging Markets:** RPM and Pairs Trading shine when trends are absent.
3. **AI Learns Market Intuition:** The TDQN agent learned to behave similarly to Buy & Hold in strong trends and reduce exposure in uncertainty.

7.6 Future Work

To further bridge the gap between academic theory and industry practice, we propose several extensions:

7.6.1 Model Improvements

1. **Transformer Models:** Replacing the Feedforward Network with a Transformer architecture to better capture long-term dependencies in price history.
2. **Multi-Agent RL:** Training multiple agents that specialize in different market regimes.
3. **Hierarchical RL:** Using a meta-controller to switch between strategies based on detected regime.

7.6.2 Feature Engineering

1. **Sentiment Analysis:** Incorporating NLP-based sentiment from financial news and social media.
2. **Alternative Data:** Using satellite imagery, credit card data, or web traffic as additional features.
3. **Cross-Asset Features:** Including bond yields, VIX, and other market indicators.

7.6.3 Risk Management

1. **Dynamic Position Sizing:** Adjusting position sizes based on volatility regime.
2. **Portfolio Optimization:** Using mean-variance optimization to combine multiple strategies.
3. **Tail Risk Hedging:** Adding options strategies for downside protection.

7.6.4 Practical Implementation

1. **Live Trading:** Connecting the Python Engine to a broker API (e.g., Alpaca or Interactive Brokers) for paper trading.
2. **Real-Time Data:** Implementing streaming data feeds for live execution.
3. **Monitoring Dashboard:** Building a real-time monitoring system for strategy performance.

7.7 Final Thoughts

The issue of Algorithmic Trading is not resolved. It is an arms race in which the most superior strategies are developed, eventually get arbitrated to death. In this project, it can be seen that AI offers a powerful new weapon, but the essentials of finance—Risk Management, Diversification, and Momentum—do not lose their usefulness.

A hybrid between human intuition is likely to make the best Algo Trader of the future: strategy design and risk control plus AI implementation to detect patterns and fast adaptation.

As the well known trader Jesse Livermore once asserted: "The game of speculation is the most uniformly fascinating game in the world. But it is not a game for the fool, the mindlessly lazy, the individual of lower temperament, or the adventurer of get-rich-quick. They will die poor."

This project has demonstrated that with proper analysis, testing, and proper risk management, algorithmic trading could be a wealth creating tool. But discipline, patience and continuous learning are some of the qualities which success needs, but no algorithm can fully replace.

7.8 Acknowledgments

We acknowledge the following resources that made this project possible:

- Yahoo Finance for historical price data
- The authors of the four papers that provided theoretical foundations
- The Python open-source community for excellent libraries
- The Stable-Baselines3 team for RL implementations

7.9 Declarations

Data Availability: All data used in this study is publicly available from Yahoo Finance.

Code Availability: Complete source code is provided in the Appendix.

Appendix A

Source Code Listings

The following sections contain the source code implemented for this study. The code is written in Python 3.8+ and relies on libraries such as `pandas`, `numpy`, `ta` (Technical Analysis), and `stable-baselines3` for reinforcement learning.

A.1 Project Structure

The codebase is organized as follows:

```
1 AlgoTrading_Project/
2 |-- data/                # CSV data files
3 |   |-- AAPL.csv
4 |   |-- GOOGL.csv
5 |   |-- MSFT.csv
6 |
7 |-- strategies/          # Trading strategy implementations
8 |   |-- __init__.py
9 |   |-- base_strategy.py # Abstract base class
10 |  |-- buy_hold.py       # Benchmark strategy
11 |  |-- momentum.py       # Jegadeesh & Titman (1993)
12 |  |-- perfect_order.py  # Technical Analysis implementation
13 |  |-- pairs_trading.py  # H gg (2023) implementation
14 |  |-- rpmm.py           # H gg (2023) implementation
15 |  |-- tdqn_agent.py     # Th ate & Ernst (2020) implementation
16 |  |-- data_augmentation.py # Trajectory generation
17 |
18 |-- backtester/          # Backtesting engine
19 |   |-- engine.py        # Main simulation loop
20 |   |-- performance.py   # Metrics calculation
21 |   |-- risk_manager.py  # Position sizing
22 |
23 |-- models/              # Saved ML models
24 |   |-- tdqn_model.zip
25 |
26 |-- results/             # Output files
27 |   |-- comparison_metrics.csv
28 |   |-- equity_curves.png
29 |
30 |-- main.py              # Entry point
31 |-- run_comparison.py     # Run all strategies
32 |-- download_data.py     # Data acquisition
```

```
33 |-- requirements.txt          # Dependencies
```

Listing A.1: Project Directory Structure

A.2 Base Strategy Interface

All strategies inherit from this abstract base class:

```
1 from abc import ABC, abstractmethod
2
3 class BaseStrategy(ABC):
4     """
5     Abstract Base Class for all trading strategies.
6     Ensures compatibility with the Backtester Engine.
7     """
8
9     def __init__(self, name):
10         self.name = name
11
12     @abstractmethod
13     def generate_signal(self, current_date, data_dict,
14                        current_positions, portfolio_value):
15         """
16         Generates trading signals for the current date.
17
18         Args:
19             current_date (pd.Timestamp): The current simulation time.
20             data_dict (dict): Dictionary of all DataFrames available.
21                               Strategy should query data_dict[symbol].
22             loc[:current_date]
23             current_positions (dict): {symbol: shares}
24             portfolio_value (float): Total Net Worth (Cash + Stock)
25
26         Returns:
27             list: List of order dictionaries.
28                 Example: [{'symbol': 'AAPL', 'action': 'buy', '
29                            cash_amount': 5000}]
30         """
31     pass
```

Listing A.2: Base Strategy Class

A.3 Backtester Engine

The core event-driven backtesting engine:

```
1 import pandas as pd
2 import numpy as np
3 from .risk_manager import RiskManager
4 from .performance import Performance
5
6 class BacktesterEngine:
7     """
8     Event-driven backtester engine.
9     Refactored for Multi-Asset support.
```

```

10     """
11
12     def __init__(self, initial_balance=10000.0, commission_percent
13     =0.001):
14         self.initial_balance = initial_balance
15         self.current_balance = initial_balance
16         self.commission_percent = commission_percent
17
18         # Positions: {'AAPL': 10, 'GOOGL': 5}
19         self.positions = {}
20         self.history = []
21
22         self.risk_manager = RiskManager()
23
24     def run(self, data_dict, strategy):
25         """
26         Runs the backtest over a portfolio of assets.
27
28         Args:
29             data_dict (dict): { 'Symbol': pd.DataFrame }.
30                             Each DF must have 'close' and datetime
31                             index.
32             strategy (BaseStrategy): Strategy instance.
33
34         Returns:
35             dict: Performance metrics.
36             pd.DataFrame: History log.
37         """
38         print(f"Starting Multi-Asset Backtest for: {strategy.name}")
39         self.current_balance = self.initial_balance
40         self.positions = {}
41         self.history = []
42
43         # 1. Align Data
44         # Get the union of all indices to create a master timeline
45         all_dates = pd.Index([])
46         for symbol, df in data_dict.items():
47             # Ensure index is datetime
48             if not isinstance(df.index, pd.DatetimeIndex):
49                 print(f"Warning: Index for {symbol} is not
50 DatetimeIndex. Attempting convert.")
51                 df.index = pd.to_datetime(df.index)
52                 all_dates = all_dates.union(df.index)
53
54         all_dates = all_dates.sort_values().unique()
55
56         print(f"Timeline established: {len(all_dates)} trading days.")
57
58         # 2. Main Event Loop
59         for current_date in all_dates:
60
61             # Construct the "Market State" for this date
62             # It contains the current row for every asset (if it exists
63             today)
64             # and potentially past history
65
66             # Optimization:
67             # Passing full history dict every tick is heavy.

```

```

64         # We'll pass a lightweight object or rely on the strategy
to look up data
65         # provided we give it the current_date.
66
67         # Let's pass the global data_dict and the current index
pointer/date
68         # The strategy should know how to slice it.
69
70         # 2.1 Calculate Portfolio Value (Mark-to-Market)
71         holdings_value = 0.0
72         portfolio_snapshot = {}
73
74         available_prices = {}
75
76         for symbol, shares in self.positions.items():
77             df = data_dict.get(symbol)
78             if df is not None:
79                 # Find price at or before current date
80                 # Using 'asof' or simply reindexing beforehand is
faster, but this is safe
81                 try:
82                     # Exact match
83                     if current_date in df.index:
84                         price = df.loc[current_date]['close']
85                     else:
86                         # Forward fill logic (use yesterday's price
)
87                         # For backtesting, we strictly use "known"
prices.
88                         # If no trade today, maybe price didn't
change or exchange closed.
89                         # We'll lookup nearest past date
90                         idx_loc = df.index.get_indexer([
current_date], method='pad')[0]
91                         if idx_loc != -1:
92                             price = df.iloc[idx_loc]['close']
93                         else:
94                             price = 0.0 # Asset didn't exist yet?
95
96                         holdings_value += shares * price
97                         available_prices[symbol] = price
98
99                 except KeyError:
100                     pass
101
102         total_value = self.current_balance + holdings_value
103
104         # 2.2 Get Strategy Signals
105         # Strategy receives: available market data up to this point
106         # For efficiency, we just pass the date and the full dict,
strategy handles lookback
107
108         signals = strategy.generate_signal(current_date, data_dict,
self.positions, total_value)
109
110         # Signal format expected:
111         # [

```

```

112         # {'symbol': 'AAPL', 'action': 'buy', 'amount': 1000 (
cash) or 'shares': 10},
113         # {'symbol': 'MSFT', 'action': 'sell', 'percent': 1.0}
114         # ]
115
116     # 2.3 Execute Signals
117     if signals:
118         for order in signals:
119             symbol = order.get('symbol')
120             action = order.get('action')
121
122             df = data_dict.get(symbol)
123             if df is None: continue
124
125             # Get execution price (Close of today, assuming we
trade at Close)
126             # Realistically: Trade at Next Open. For this
academic exercise, Close is standard.
127             if current_date in df.index:
128                 exec_price = df.loc[current_date]['close']
129             else:
130                 continue # Can't trade if no price
131
132             if action == 'buy':
133                 # Determine size (Cash amount or Share count)
134                 cash_alloc = order.get('cash_amount', 0)
135
136                 # Apply risk management if not explicitly sized
?
137                 # For momentum, usually we allocate "10% of
portfolio".
138
139                 if cash_alloc > 0:
140                     # Commission
141                     # cash_needed = trade_value + comm
142                     # trade_value = shares * price
143                     # comm = trade_value * comm_pct
144                     # cost = trade_value * (1 + comm_pct)
145
146                     # max_trade_value = cash_alloc / (1 +
comm_pct)
147                     shares_to_buy = (cash_alloc / (1 + self.
commission_percent)) // exec_price
148
149                     cost = shares_to_buy * exec_price
150                     comm = cost * self.commission_percent
151                     total_cost = cost + comm
152
153                     if total_cost <= self.current_balance and
shares_to_buy > 0:
154                         self.current_balance -= total_cost
155                         self.positions[symbol] = self.positions
.get(symbol, 0) + shares_to_buy
156                         # print(f"[{current_date.date()}] BUY {
symbol} {shares_to_buy} @ {exec_price:.2f}")
157
158                     elif action == 'sell':
159                         # Sell all or specific amount

```

```

160         current_shares = self.positions.get(symbol, 0)
161         if current_shares > 0:
162             revenue = current_shares * exec_price
163             comm = revenue * self.commission_percent
164
165             self.current_balance += (revenue - comm)
166             del self.positions[symbol]
167             # print(f"[{current_date.date()}] SELL {
symbol} {current_shares} @ {exec_price:.2f}")
168
169         elif action == 'close_all':
170             for sym, shares in list(self.positions.items()
):
171                 if shares > 0:
172                     # Need price
173                     if sym in data_dict and current_date
in data_dict[sym].index:
174                         p = data_dict[sym].loc[
current_date]['close']
175                         rev = shares * p
176                         comm = rev * self.
commission_percent
177                         self.current_balance += (rev -
comm)
178                         self.positions = {}
179
180         # 3. Record History
181         self.history.append({
182             'Date': current_date,
183             'Cash': self.current_balance,
184             'TotalValue': total_value
185         })
186
187         # Final Report
188         history_df = pd.DataFrame(self.history).set_index('Date')
189         metrics = Performance.calculate_metrics(history_df)
190
191         return metrics, history_df

```

Listing A.3: Event-Driven Backtest Engine

A.4 Performance Metrics Module

Calculates industry-standard performance metrics:

```

1 import numpy as np
2 import pandas as pd
3
4 class Performance:
5     """
6     Calculates performance metrics for trading strategies.
7     References:
8     - Paper 2 (Geometric Mean, Sharpe Ratio)
9     - Standard Financial Metrics
10    """
11
12    @staticmethod

```

```

13     def calculate_sharpe_ratio(returns, risk_free_rate=0.0,
14                                periods_per_year=252):
15         """
16         Calculates the Annualized Sharpe Ratio of a returns series.
17
18         Args:
19             returns (pd.Series or np.array): Period returns (e.g.,
20             daily).
21             risk_free_rate (float): Annualized risk-free rate.
22             periods_per_year (int): Trading periods per year (252 for
23             daily).
24
25         Returns:
26             float: Annualized Sharpe Ratio.
27         """
28         if len(returns) < 2:
29             return 0.0
30
31         # Convert annual risk-free rate to per-period rate
32         rf_per_period = risk_free_rate / periods_per_year
33
34         excess_returns = returns - rf_per_period
35         mean_excess_return = np.mean(excess_returns)
36         std_dev = np.std(excess_returns, ddof=1)
37
38         if std_dev == 0:
39             return 0.0
40
41         # Annualize the Sharpe Ratio
42         # Sharpe = (mean_return * sqrt(periods)) / (std * sqrt(periods))
43         ) = mean/std * sqrt(periods)
44         daily_sharpe = mean_excess_return / std_dev
45         annualized_sharpe = daily_sharpe * np.sqrt(periods_per_year)
46
47         return annualized_sharpe
48
49     @staticmethod
50     def calculate_max_drawdown(equity_curve):
51         """
52         Calculates the Maximum Drawdown of an equity curve.
53
54         Args:
55             equity_curve (pd.Series or np.array): Series of portfolio
56             values/prices.
57
58         Returns:
59             float: Maximum percentage drop from a peak.
60         """
61         if len(equity_curve) < 1:
62             return 0.0
63
64         # Convert to numpy for speed
65         values = np.array(equity_curve)
66
67         # Calculate running maximum
68         running_max = np.maximum.accumulate(values)
69
70         # Calculate drawdown for each point

```

```

66         drawdowns = (values - running_max) / running_max
67
68         # Max drawdown is the minimum value (most negative)
69         max_dd = np.min(drawdowns)
70
71         return max_dd
72
73     @staticmethod
74     def calculate_geometric_mean_return(returns):
75         """
76         Calculates Geometric Mean Return (AGR) as suggested in Paper 2.
77         AGR = (Product(1 + r))^(1/n) - 1
78
79         Args:
80             returns (pd.Series or np.array): Series of fractional
81             returns (e.g., 0.01 for 1%).
82
83         Returns:
84             float: Geometric Mean Return per period.
85         """
86         if len(returns) < 1:
87             return 0.0
88
89         # Add 1 to returns to get growth factors
90         growth_factors = 1 + returns
91
92         # Ensure no negative values if returns < -100% (bankruptcy)
93         if np.any(growth_factors <= 0):
94             return -1.0 # Lost everything
95
96         # Calculate geometric mean
97         # Using exp(mean(log(x))) is numerically more stable for large
98         n
99         log_growth = np.log(growth_factors)
100         mean_log_growth = np.mean(log_growth)
101         geometric_mean = np.exp(mean_log_growth) - 1
102
103         return geometric_mean
104
105     @staticmethod
106     def calculate_metrics(history_df):
107         """
108         Computes a comprehensive metrics dictionary from a history
109         DataFrame.
110         Expected columns: 'TotalValue' (indexed by time).
111         """
112         if history_df.empty:
113             return {}
114
115         equity_curve = history_df['TotalValue']
116
117         # Calculate returns
118         returns = equity_curve.pct_change().dropna()
119
120         total_return = (equity_curve.iloc[-1] / equity_curve.iloc[0]) -
121         1
122
123         sharpe = Performance.calculate_sharpe_ratio(returns)
124         max_dd = Performance.calculate_max_drawdown(equity_curve)

```



```

120     geom_mean = Performance.calculate_geometric_mean_return(returns
121     )
122     # Annualized volatility (assuming daily data, 252 days)
123     volatility = returns.std() * np.sqrt(252) if len(returns) > 1
124     else 0.0
125
126     return {
127         "Total Return": total_return,
128         "Sharpe Ratio": sharpe,
129         "Max Drawdown": max_dd,
130         "Geometric Mean Return": geom_mean,
131         "Annualized Volatility": volatility
132     }

```

Listing A.4: Performance Metrics

A.5 Buy and Hold Strategy (Benchmark)

The simplest strategy for baseline comparison:

```

1 from .base_strategy import BaseStrategy
2
3 class BuyHoldStrategy(BaseStrategy):
4     """
5     Simplest strategy: Buy at the beginning and hold forever.
6     Updated for Multi-Asset Engine.
7     """
8
9     def __init__(self):
10         super().__init__("Buy & Hold")
11         self.bought = False
12
13     def generate_signal(self, current_date, data_dict,
14                       current_positions, portfolio_value):
15         """
16         Returns buy signals for all available assets on the first tick,
17         then nothing.
18         """
19         if not self.bought:
20             self.bought = True
21
22             # Buy everything available equally
23             orders = []
24             available_symbols = [s for s, df in data_dict.items() if
25                               current_date in df.index]
26
27             if not available_symbols:
28                 self.bought = False # Try again next tick unique case
29                 return []
30
31             allocation = portfolio_value * 0.95 / len(available_symbols
32 )
33
34             for sym in available_symbols:
35                 orders.append({
36                     'symbol': sym,

```

```

33         'action': 'buy',
34         'cash_amount': allocation
35     })
36
37     return orders
38
39     return []

```

Listing A.5: Buy and Hold Implementation

A.6 Momentum Strategy (Jegadeesh & Titman, 1993)

Implementation of the momentum strategy from Jegadeesh & Titman (1993):

```

1 from .base_strategy import BaseStrategy
2 import pandas as pd
3 import numpy as np
4
5 class MomentumStrategy(BaseStrategy):
6     """
7     Implements 'Returns to Buying Winners and Selling Losers' (
8     Jegadeesh & Titman, 1993).
9
10    Parameters:
11    - J (Lookback Period): 12 months
12    - K (Holding Period): 3 months
13    - Skip Week: Exclude last week updates to avoid microstructure
14    noise.
15    - January Effect: Avoid long positions in January.
16    """
17
18    def __init__(self, j_months=12, k_months=3, top_percentile=0.1):
19        super().__init__("Momentum (J=12, K=3)")
20        self.j_months = j_months
21        self.k_months = k_months
22        self.top_percentile = top_percentile
23
24        self.rebalance_tracker = 0 # Months since last rebalance
25        self.last_rebalance_month = None
26
27    def generate_signal(self, current_date, data_dict,
28    current_positions, portfolio_value):
29        """
30        Executes momentum logic.
31        """
32        orders = []
33
34        # 1. Check if we should rebalance
35        # Logic: Rebalance at the start of the simulation, and then
36        every K months
37        # Simplified: Check if month changed. If it's a "rebalance
38        month", act.
39
40        current_month = current_date.month
41        current_year = current_date.year
42
43        # Track unique months seen to handle Rebalance Frequency K

```

```

39     # A simple approximation: Rebalance on the 1st trading day of
    the month
40     if self.last_rebalance_month == current_month:
41         return [] # Already processed this month
42
43     # It's a new month. check K period.
44     # Ideally we track robustly, but for now let's rebalance EVERY
    MONTH (K=1 sliding)
45     # Or strictly every K. Let's do K=3 fixed blocks for fidelity
    to the paper?
46     # The paper says "Overlapping portfolios" or "Fixed holding".
47     # Let's implement: Rebalance Every Month to hold the "Top
    Decile" as determined
48     # by the J-month lookback. This is a common modern
    interpretation (monthly rebalance).
49
50     self.last_rebalance_month = current_month
51
52     # 2. January Effect Filter
53     # "If Month == January: Close Long positions"
54     if current_month == 1:
55         # print(f"[{current_date.date()}] January Element: Closing
    Longs/Staying Cash.")
56         return [{'action': 'close_all'}]
57
58     # 3. Calculate Momentum for all assets
59     scores = []
60
61     for symbol, df in data_dict.items():
62         # Need history up to current_date
63         # Slicing: df.loc[:current_date]
64
65         # Optimization: use get_loc ensures we don't copy the whole
    DF
66
67         # But we need a window.
68
69         # Lookback: J=12 months approx 252 days.
70         # Skip week: 5 days.
71
72         # Let's get the price 1 week ago and 12 months ago
73
74         # Efficient lookup:
75         try:
76             # Get index location of current date
77             # We need exact index or nearest past
78             curr_idx_loc = df.index.get_indexer([current_date],
    method='pad')[0]
79             if curr_idx_loc == -1: continue
80
81             # 1 Week Ago index
82             week_ago_idx = curr_idx_loc - 5
83             # 12 Months Ago index (approx 252 days)
84             year_ago_idx = curr_idx_loc - 252
85
86             if year_ago_idx < 0: continue # Not enough history
87
88             price_now = df.iloc[curr_idx_loc]['close']
            price_1wk_ago = df.iloc[week_ago_idx]['close']

```

```

89         price_1yr_ago = df.iloc[year_ago_idx]['close']
90
91         # Momentum Return = (Price[1_Week_Ago] / Price[12
92         _Months_Ago]) - 1
93         # We use 1 week ago as the 'Recent' price to skip the
94         bid-ask bounce
95
96         momentum_score = (price_1wk_ago / price_1yr_ago) - 1
97
98         scores.append((symbol, momentum_score))
99
100     except Exception as e:
101         # Index out of bounds or data issues
102         continue
103
104     if not scores:
105         return []
106
107     # 4. Rank and Select
108     # Sort by score descending
109     scores.sort(key=lambda x: x[1], reverse=True)
110
111     num_assets = len(scores)
112     top_n = max(1, int(num_assets * self.top_percentile))
113
114     winners = [x[0] for x in scores[:top_n]]
115
116     # print(f"[{current_date.date()}] Winners: {winners}")
117
118     # 5. Generate Rebalance Orders
119     # Sell everything currently held that is NOT a winner
120     # Buy Winners equal weight
121
122     # First, sell non-winners
123     for symbol in list(current_positions.keys()):
124         if symbol not in winners:
125             orders.append({'symbol': symbol, 'action': 'sell'})
126
127     # Determine Cash Allocation per Winner
128     # We want to be fully invested in Winners
129     # Total Equity = portfolio_value
130     # Allocation per asset = Total Equity / top_n
131
132     target_alloc = portfolio_value * 0.95 / top_n # 95% invested to
133     leave buffer for comms
134
135     for win_sym in winners:
136         # If we don't have it, buy it
137         if win_sym not in current_positions:
138             orders.append({
139                 'symbol': win_sym,
140                 'action': 'buy',
141                 'cash_amount': target_alloc
142             })
143         else:
144             # Rebalance? (Optional, skip for simplicity or just
145             hold)

```

```

142         # If we already have it, we could adjust to match
target_alloc
143         # Let's Hold for now to minimize turnover costs
144         pass
145
146     return orders

```

Listing A.6: Momentum Strategy

A.7 Perfect Order Strategy (Technical Analysis)

Trend following with multiple moving averages:

```

1 from .base_strategy import BaseStrategy
2 import pandas as pd
3 import numpy as np
4 import ta
5
6 class PerfectOrderStrategy(BaseStrategy):
7     """
8     Implements 'Perfect Order' Strategy (Paper 4).
9     Trend Following with Multiple SMA confirmations and ADX.
10    """
11
12    def __init__(self):
13        super().__init__("Perfect Order (5 SMAs + ADX)")
14        self.consecutive_days = 0
15        self.last_signal = None
16        # For simplicity in this multi-asset engine, we track
consecutive days PER ASSET.
17        # But 'consecutive_days' as a single int won't work for multi-
asset.
18        self.asset_states = {} # {symbol: {'consecutive': 0, 'trend': '
none'}}
19
20    def generate_signal(self, current_date, data_dict,
current_positions, portfolio_value):
21        orders = []
22
23        # Allocate per asset (naive equal weight for now, or dynamic)
24        # Strategy says: Risk Management calculates size.
25        # Here we just trigger the entry, engine/risk_manager sizing
logic handles quantity?
26        # Our engine expects 'cash_amount' for buys.
27
28        # Max concurrent positions? Let's say 5.
29        max_positions = 5
30        current_holdings_count = len(current_positions)
31
32        for symbol, df in data_dict.items():
33            if symbol not in self.asset_states:
34                self.asset_states[symbol] = {'consecutive': 0, 'trend':
'none'}
35
36                state = self.asset_states[symbol]
37
38                # Need history for indicators

```

```

39         # SMAs need 200 days. ADX needs 14 + smoothing.
40         # Let's grab 300 days to be safe.
41
42         try:
43             # Efficient lookup of "upto now"
44             curr_loc = df.index.get_indexer([current_date], method=
'pad')[0]
45             if curr_loc < 250: continue # Not enough data
46
47             # Slice last 250 rows
48             history = df.iloc[curr_loc-250 : curr_loc+1].copy()
49
50             # Check for NaNs using 'Close'
51             if history['close'].isnull().values.any():
52                 # clean or skip
53                 history = history.dropna()
54                 if len(history) < 200: continue
55
56             # Calculate Indicators
57             # Note: Calculating full TA series every step is
inefficient for backtesting entire history loop.
58             # Optimization: In a real backtester, we calc
indicators ONCE for the whole dataframe at start.
59             # But our interface passes 'data_dict' which might be
growing.
60             # For 'BacktesterEngine' as implemented, 'data_dict'
contains FULL DFs.
61             # So we SHOULD calc indicators once in '__init__' or '
setup' if possible.
62             # However, 'generate_signal' is called daily.
63             # Let's assume we can pre-calculate if we want, but to
keep it simple and robust to "streaming",
64             # we calculate on the fly (slow) or look up pre-
calculated columns.
65
66             # Fast path: Check if 'sma_200' exists in columns. If
not, we can't assume.
67             # Let's do a lightweight calculation on the tail, or
assume user/main adds them.
68             # For this implementation, let's Calculate on the fly (
simple, but slow).
69
70             # To make it faster: Only calculate last few values? TA
lib usually computes whole series.
71             # Better approach for this loop:
72             # Retrieve the VALUES from pre-calculated columns if
they exist.
73             # If they don't, calculate them now.
74
75             # Let's perform calculation on the specific window
passed.
76
77             close = history['close']
78             high = history['high']
79             low = history['low']
80
81             # SMAs

```

```

82         sma_10 = ta.trend.sma_indicator(close, window=10).iloc
83         [-1]
84         sma_20 = ta.trend.sma_indicator(close, window=20).iloc
85         [-1]
86         sma_50 = ta.trend.sma_indicator(close, window=50).iloc
87         [-1]
88         sma_100 = ta.trend.sma_indicator(close, window=100).
89         iloc[-1]
90         sma_200 = ta.trend.sma_indicator(close, window=200).
91         iloc[-1]
92
93         # ADX
94         adx = ta.trend.ADXIndicator(high, low, close, window
95         =14).adx().iloc[-1]
96
97         # ATR (for risk management)
98         atr = ta.volatility.AverageTrueRange(high, low, close,
99         window=14).average_true_range().iloc[-1]
100
101         # 1. Check Perfect Order Conditions
102
103         # BUY: 10 > 20 > 50 > 100 > 200 AND ADX >= 20
104         is_buy_setup = (sma_10 > sma_20 > sma_50 > sma_100 >
105         sma_200) and (adx >= 20)
106
107         # SELL: 10 < 20 < 50 < 100 < 200 AND ADX >= 20
108         is_sell_setup = (sma_10 < sma_20 < sma_50 < sma_100 <
109         sma_200) and (adx >= 20)
110
111         # 2. Confirmation Logic (5 bars)
112         if is_buy_setup:
113             if state['trend'] == 'buy_setup':
114                 state['consecutive'] += 1
115             else:
116                 state['trend'] = 'buy_setup'
117                 state['consecutive'] = 1
118         elif is_sell_setup:
119             if state['trend'] == 'sell_setup':
120                 state['consecutive'] += 1
121             else:
122                 state['trend'] = 'sell_setup'
123                 state['consecutive'] = 1
124         else:
125             state['consecutive'] = 0
126             state['trend'] = 'none'
127
128         # 3. Execution Logic
129
130         # Entry Signal
131         if state['trend'] == 'buy_setup' and state['consecutive
132         '] >= 5:
133
134             if symbol not in current_positions:
135                 if current_holdings_count < max_positions:
136                     # Position Sizing
137                     # Risk 2% of Account
138                     # Stop Loss = 2 * ATR
139                     stop_loss = close.iloc[-1] - (2 * atr)
140                     stop_dist = close.iloc[-1] - stop_loss

```

```

130
131         risk_amt = portfolio_value * 0.02
132
133         # If stop dist is tiny, cap size
134         if stop_dist > 0:
135             target_cash = (risk_amt / stop_dist) *
136
137         close.iloc[-1]
138
139         # Cap at 20% of portfolio
140         max_alloc = portfolio_value * 0.20
141         target_cash = min(target_cash,
142
143         max_alloc)
144
145         orders.append({
146             'symbol': symbol,
147             'action': 'buy',
148             'cash_amount': target_cash,
149             'stop_loss': stop_loss
150         })
151
152         # Exit Logic (Simple Trend Breakdown)
153         # If we hold it, and setup breaks, do we sell?
154         # Paper 4 suggests Trailing Stop or Trend Reversal.
155         # Let's use the ATR Stop Loss implicit in "Risk Manager
156         "
157         # BUT the engine handles "Risk Manager" for quantity,
158         not exit monitoring yet?
159         # A real engine monitors stops. Our simple engine doesn
160         't have "pending stop orders".
161         # We must self-monitor.
162
163         if symbol in current_positions:
164             # Check Soft Stop
165             # We don't know our entry price here easily without
166             position tracking enhancement.
167             # Simplified: If Trend reverses (Sell Setup
168             confirmed), close Longs.
169
170             if state['trend'] == 'sell_setup' and state['
171             consecutive'] >= 2: # Fast exit
172                 orders.append({'symbol': symbol, 'action': '
173             sell'})
174
175         except Exception as e:
176             # print(f"Error {symbol}: {e}")
177             continue
178
179         return orders

```

Listing A.7: Perfect Order Strategy

A.8 Pairs Trading Strategy (Hägg, 2023)

Statistical arbitrage using spread trading, based on Hägg (2023):

```

1 from .base_strategy import BaseStrategy
2 import pandas as pd

```



```

3 import numpy as np
4
5 class PairsTradingStrategy(BaseStrategy):
6     """
7     Implements 'Pairs Trading' (Statistical Arbitrage) from Paper 2.
8     Uses Bollinger Bands on the spread (Price A / Price B).
9     """
10
11     def __init__(self, ticker_a='GOOGL', ticker_b='MSFT', window=20,
12                  std_dev=2):
13         super().__init__(f"Pairs Trading ({ticker_a}/{ticker_b})")
14         self.ticker_a = ticker_a
15         self.ticker_b = ticker_b
16         self.window = window
17         self.std_dev = std_dev
18         self.in_position = False # 'long_spread' (Long A, Short B) or '
19         short_spread' (Short A, Long B) or False
20
21     def generate_signal(self, current_date, data_dict,
22                       current_positions, portfolio_value):
23         orders = []
24
25         # Ensure both assets exist
26         if self.ticker_a not in data_dict or self.ticker_b not in
27         data_dict:
28             return []
29
30         df_a = data_dict[self.ticker_a]
31         df_b = data_dict[self.ticker_b]
32
33         # Need history for indicators
34         # Window size 20 is small, let's grab 50 to be safe
35         try:
36             loc_a = df_a.index.get_indexer([current_date], method='pad'
37             ) [0]
38             loc_b = df_b.index.get_indexer([current_date], method='pad'
39             ) [0]
40
41             if loc_a < 30 or loc_b < 30: return []
42
43             # Align history approximately?
44             # Easiest way: Get the prices for the last N dates
45             available in the master timeline
46             # Engine doesn't pass master timeline history easily
47             without full scan.
48             # Let's slice the DataFrames individually assuming they are
49             somewhat aligned or we merge.
50             # Best practice for Pairs: Merge on Date.
51
52             # Slice last 50 rows from both, merge to ensure date
53             alignment
54             slice_a = df_a.iloc[loc_a-50 : loc_a+1]['close'].rename(
55             self.ticker_a)
56             slice_b = df_b.iloc[loc_b-50 : loc_b+1]['close'].rename(
57             self.ticker_b)
58
59             merged = pd.concat([slice_a, slice_b], axis=1).dropna()

```

```

49         if len(merged) < self.window: return []
50
51         # Calculate Spread
52         # Paper 2: Spread = Price A / Price B (Ratio) or Price A -
Multiplier * Price B (Cointegration)
53         # Paper text says "Price A / Price B" or similar simple
ratio for basic pairs.
54         spread = merged[self.ticker_a] / merged[self.ticker_b]
55
56         # Bollinger Bands on Spread
57         rolling_mean = spread.rolling(window=self.window).mean()
58         rolling_std = spread.rolling(window=self.window).std()
59
60         upper_band = rolling_mean + (self.std_dev * rolling_std)
61         lower_band = rolling_mean - (self.std_dev * rolling_std)
62
63         current_spread = spread.iloc[-1]
64         curr_upper = upper_band.iloc[-1]
65         curr_lower = lower_band.iloc[-1]
66         curr_mean = rolling_mean.iloc[-1]
67
68         # Logic
69         # If Spread > Upper -> Spread is too high (A is expensive
relative to B) -> Short A, Long B
70         # If Spread < Lower -> Spread is too low (A is cheap
relative to B) -> Long A, Short B
71         # Exit when Spread returns to Mean
72
73         # Current holdings: we check self.in_position or query
current_positions
74
75         has_a = current_positions.get(self.ticker_a, 0)
76         has_b = current_positions.get(self.ticker_b, 0)
77
78         # Define State from positions (simplified)
79         # Long Spread: Long A, Short B (or just Long A if short
restricted)
80         # Short Spread: Short A, Long B
81
82         # Entry Logic
83         if not self.in_position:
84             if current_spread > curr_upper:
85                 # Signal: Short Spread (Short A, Long B)
86                 # Allocation: 50% capital each leg? Or 100% total?
87                 half_capital = portfolio_value * 0.45
88
89                 orders.append({'symbol': self.ticker_a, 'action': '
sell'}) # Short A (engine assumes sell=short if 0 held? Need
explicit short support or just sell what we don't have)
90                 # Note: Engine 'sell' currently just reduces
position. If we seek to Short, we need negative position support.
91                 # Assuming Engine supports negative quantity
positions implicitly if we sell from 0?
92                 # Engine code: 'current_shares = positions.get(
symbol, 0); if current_shares > 0...'
93                 # Ah, basic engine prevented selling if shares=0.
94                 # CRITICAL: We need to enable Shorting in Engine
for Pairs Trading.

```

```

95         # Workaround for now: Long-Only Pairs? (Long B when
        Spread High, Long A when Spread Low)
96         # That is valid. "Switching" strategy.
97
98         # Let's implement Long-Only "Switching" for safety
        if Shorting is complex.
99         # "Short A, Long B" -> Just Buy B (and Sell A if
        held)
100
101         orders.append({'symbol': self.ticker_a, 'action': '
        sell'}) # Close A if held
102         orders.append({'symbol': self.ticker_b, 'action': '
        buy', 'cash_amount': half_capital * 2}) # Buy B
103         self.in_position = 'short_spread' # Actually just
        Long B
104
105         elif current_spread < curr_lower:
106         # Signal: Long Spread (Long A, Short B) -> Just Buy
        A
107         half_capital = portfolio_value * 0.45
108
109         orders.append({'symbol': self.ticker_b, 'action': '
        sell'}) # Close B if held
110         orders.append({'symbol': self.ticker_a, 'action': '
        buy', 'cash_amount': half_capital * 2}) # Buy A
111         self.in_position = 'long_spread' # Actually just
        Long A
112
113         # Exit Logic (Mean Reversion)
114         elif self.in_position:
115         # Check for Mean Reversion (crossing the moving average
        )
116         # If we were Long Spread (Held A), and spread goes >
        Mean, we exit? or wait for Upper?
117         # Usually exit at Mean.
118
119         if self.in_position == 'long_spread' and current_spread
        >= curr_mean:
120         orders.append({'symbol': self.ticker_a, 'action': '
        sell'}) # Close A
121         self.in_position = False
122
123         elif self.in_position == 'short_spread' and
        current_spread <= curr_mean:
124         orders.append({'symbol': self.ticker_b, 'action': '
        sell'}) # Close B
125         self.in_position = False
126
127     except Exception as e:
128         # print(f"Pairs Error: {e}")
129         pass
130
131     return orders

```

Listing A.8: Pairs Trading Strategy

A.9 RPMM Strategy (Hägg, 2023)

Reservation Price Min-Max algorithm, based on Hägg (2023):

```

1 from .base_strategy import BaseStrategy
2 import numpy as np
3
4 class RPMMStrategy(BaseStrategy):
5     """
6     Implements 'Modified RPMM' (Reservation Price) from Paper 2.
7     """
8
9     def __init__(self, window=10):
10         super().__init__("RPMM (Modified)")
11         self.window = window
12
13     def generate_signal(self, current_date, data_dict,
14                        current_positions, portfolio_value):
15         orders = []
16
17         # Iterate over all assets (RPMM is single-asset logic applied
18         # to many)
19         for symbol, df in data_dict.items():
20             try:
21                 # Need history
22                 loc = df.index.get_indexer([current_date], method='pad'
23 ) [0]
24
25                 if loc < self.window: continue
26
27                 # Get Window
28                 history = df.iloc[loc - self.window : loc + 1]
29
30                 # Calculate M (Max) and m (Min) in window
31                 # 'High' and 'Low' columns preferred? Or just Close?
32                 # Paper uses High/Low of the period.
33
34                 if 'high' in history.columns and 'low' in history.
35 columns:
36                     M = history['high'].max()
37                     m = history['low'].min()
38                 else:
39                     M = history['close'].max()
40                     m = history['close'].min()
41
42                 # Reservation Price q*
43                 q_star = np.sqrt(M * m)
44
45                 current_price = history.iloc[-1]['close']
46
47                 has_position = symbol in current_positions
48
49                 # Rule:
50                 # Buy if Price <= q*
51                 # Sell if Price >= q*
52
53                 cash_alloc = portfolio_value * 0.95 / len(data_dict) #
54 Split capital

```

```

50         if current_price <= q_star and not has_position:
51             orders.append({
52                 'symbol': symbol,
53                 'action': 'buy',
54                 'cash_amount': cash_alloc
55             })
56
57         elif current_price >= q_star and has_position:
58             orders.append({
59                 'symbol': symbol,
60                 'action': 'sell'
61             })
62
63     except Exception as e:
64         continue
65
66     return orders

```

Listing A.9: RPMM Strategy

A.10 TDQN Agent (Théate & Ernst, 2020)

Deep Reinforcement Learning trading agent, based on Théate & Ernst (2020):

```

1  from .base_strategy import BaseStrategy
2  from .data_augmentation import DataAugmentation
3  import pandas as pd
4  import numpy as np
5  import os
6  import random
7  import gymnasium as gym
8  from gymnasium import spaces
9  from stable_baselines3 import DQN
10 from stable_baselines3.common.vec_env import DummyVecEnv
11
12 # --- Custom Trading Env for TDQN ---
13 class TDQNTradingEnv(gym.Env):
14     """
15     Custom Trading Environment for TDQN Agent.
16     Supports training on multiple trajectories (e.g. artificial slices)
17     .
18     On reset, picks a random trajectory.
19     """
20     def __init__(self, data_source, initial_balance=10000):
21         super(TDQNTradingEnv, self).__init__()
22
23         # data_source can be a single DF or a list of DFs
24         self.trajectories = data_source if isinstance(data_source, list)
25         else [data_source]
26         self.initial_balance = initial_balance
27
28         # Action space: 0=Hold, 1=Buy, 2=Sell
29         self.action_space = spaces.Discrete(3)
30
31         # Observation space: 10 features
32         self.observation_space = spaces.Box(
33             low=-np.inf, high=np.inf, shape=(10,), dtype=np.float32

```

```

32     )
33
34     # Current episode state
35     self.df = None
36     self.current_step = 0
37
38     self.reset()
39
40     def reset(self, seed=None, options=None):
41         super().reset(seed=seed)
42
43         # Pick a random trajectory for this episode
44         self.df = random.choice(self.trajectories)
45
46         self.balance = self.initial_balance
47         self.shares_held = 0
48         self.net_worth = self.initial_balance
49         self.current_step = 0
50
51         return self._next_observation(), {}
52
53     def _next_observation(self):
54         obs = np.zeros(10, dtype=np.float32)
55         if self.current_step < len(self.df):
56             row = self.df.iloc[self.current_step]
57             # Simple Normalization
58             # Features: Close, Volume, Open, High, Low, Balance, Shares
, NetWorth
59             # We strictly need relative inputs for generalization.
60             # Using % change or log returns is better, but obeying
simplistic Paper 1 idea:
61
62             c = row['close']
63             obs[0] = c / 1000.0
64             obs[1] = row['volume'] / 1e6 if 'volume' in row else 0
65             obs[2] = self.shares_held
66             obs[3] = self.balance / 10000.0
67             # ... fill others as needed
68
69             return obs
70
71     def step(self, action):
72         # Execute Action
73         current_price = self.df.iloc[self.current_step]['close']
74
75         # 1=Buy, 2=Sell
76         if action == 1:
77             # Invest 10%
78             cost = self.balance * 0.1
79             if cost > current_price:
80                 n_shares = int(cost / current_price)
81                 self.balance -= n_shares * current_price
82                 self.shares_held += n_shares
83         elif action == 2:
84             # Sell all
85             if self.shares_held > 0:
86                 self.balance += self.shares_held * current_price
87                 self.shares_held = 0

```

```

88
89     # Update Net Worth
90     self.net_worth = self.balance + (self.shares_held *
current_price)
91
92     # Reward: Change in Net Worth (Profit)
93     # Scaled for stability
94     prev_net = (self.initial_balance) # Simplification, ideally
track prev step
95     # Let's just reward total return at end? or step return?
96     # SB3 likes dense rewards.
97     reward = (self.net_worth - self.initial_balance) / self.
initial_balance
98
99     self.current_step += 1
100     terminated = self.current_step >= len(self.df) - 1
101
102     return self._next_observation(), reward, terminated, False, {}
103
104
105 class TDQNAgent(BaseStrategy):
106     """
107     RL Agent Strategy using Stable Baselines 3 DQN.
108     """
109     def __init__(self, model_path="models/tdqn_model"):
110         super().__init__("TDQN (AI Agent)")
111         self.model_path = model_path
112         self.model = None
113
114     def train(self, data_dict, total_timesteps=10000):
115         print("Training TDQN Agent with Data Augmentation...")
116
117         # Generate Artificial Trajectories from ALL available assets
118         all_trajectories = []
119         for symbol, df in data_dict.items():
120             # Paper 1 suggests splitting history into windows
121             trajs = DataAugmentation.generate_trajectories(df,
window_size=252, step_size=60)
122             all_trajectories.extend(trajs)
123
124         print(f"Training on {len(all_trajectories)} augmented
trajectories.")
125
126         env = DummyVecEnv([lambda: TDQNTradingEnv(all_trajectories)])
127
128         self.model = DQN("MlpPolicy", env, verbose=1)
129         self.model.learn(total_timesteps=total_timesteps)
130
131         os.makedirs(os.path.dirname(self.model_path), exist_ok=True)
132         self.model.save(self.model_path)
133         print(f"Model saved to {self.model_path}")
134
135     def generate_signal(self, current_date, data_dict,
current_positions, portfolio_value):
136         """
137         Inference: Uses the model to predict action.
138         """
139         # If no model, random or hold

```

```

140     if self.model is None:
141         # Try load
142         try:
143             self.model = DQN.load(self.model_path)
144         except:
145             return [] # No model yet
146
147     orders = []
148     # For each asset, construct observations and predict
149     for symbol, df in data_dict.items():
150         # Get data locally
151         # Construct state (must match Env state def)
152         obs = np.zeros(10, dtype=np.float32)
153         # fill obs ...
154
155         action, _ = self.model.predict(obs, deterministic=True)
156
157         # Map action to Order
158         # 0=Hold, 1=Buy, 2=Sell
159         if action == 1:
160             orders.append({'symbol': symbol, 'action': 'buy', '
cash_amount': portfolio_value * 0.1})
161         elif action == 2:
162             orders.append({'symbol': symbol, 'action': 'sell'})
163
164     return orders

```

Listing A.10: TDQN Agent with Custom Environment

A.11 Data Augmentation Module

Artificial trajectory generation for RL training:

```

1 import pandas as pd
2 import numpy as np
3
4 class DataAugmentation:
5     """
6     Implements 'Artificial Trajectory Generation' from Paper 1.
7     Splits long financial time series into shorter, overlapping
8     episodes
9     to provide more training data for the RL agent.
10    """
11
12    @staticmethod
13    def generate_trajectories(df, window_size=252, step_size=20):
14        """
15        Slices a dataframe into multiple overlapping windows.
16
17        Args:
18            df (pd.DataFrame): The history source.
19            window_size (int): Length of each trajectory (e.g., 1 year)
20            .
21            step_size (int): How many distinct days to shift for next
22            window.
23
24        Returns:

```



```

22         list of pd.DataFrame: The slices.
23         """
24         trajectories = []
25
26         if len(df) < window_size:
27             return [df]
28
29         for i in range(0, len(df) - window_size + 1, step_size):
30             slice_df = df.iloc[i : i + window_size].copy()
31             # Normalize? Paper suggests normalizing each trajectory
independently
32             # so the agent learns patterns, not absolute price levels.
33             # We'll leave normalization to the Environment/Agent logic,
34             # here we just slice.
35             trajectories.append(slice_df)
36
37         print(f"Generated {len(trajectories)} trajectories from data
length {len(df)}")
38         return trajectories

```

Listing A.11: Data Augmentation

A.12 Main Comparison Script

Script to run all strategies and generate results:

```

1 import pandas as pd
2 import matplotlib.pyplot as plt
3 import os
4 from main import load_data
5 from backtester.engine import BacktesterEngine
6 from backtester.performance import Performance
7
8 # Import Strategies
9 from strategies.buy_hold import BuyHoldStrategy
10 from strategies.momentum import MomentumStrategy
11 from strategies.perfect_order import PerfectOrderStrategy
12 from strategies.pairs_trading import PairsTradingStrategy
13 from strategies.rpmm import RPMMStrategy
14 from strategies.tdqn_agent import TDQNAgent
15
16 def run_comparison():
17     print("Loading Data...")
18     # Load all data or a subset?
19     # Let's load the big 3 for general strategies
20     tickers = ['AAPL', 'GOOGL', 'MSFT']
21     data_dict = load_data(selected_symbols=tickers)
22
23     # Define Strategies to Test
24     strategies = {
25         "Buy & Hold": BuyHoldStrategy(),
26         "Momentum (Paper 3)": MomentumStrategy(),
27         "Perfect Order (Paper 4)": PerfectOrderStrategy(),
28         "TDQN AI (Paper 1)": TDQNAgent(), # Will load pre-trained model
if exists
29         "RPMM (Paper 2)": RPMMStrategy()
30     }

```

```

31
32 # Pairs Trading needs specific pair
33 # We can run it on the same data dict if it contains the pair
34 strategies["Pairs Trading (Paper 2)"] = PairsTradingStrategy(
    ticker_a='GOOGL', ticker_b='MSFT')
35
36 results = []
37 equity_curves = {}
38
39 print(f"Running Backtests for {len(strategies)} strategies...")
40
41 for name, strategy in strategies.items():
42     print(f"Running {name}...")
43     try:
44         engine = BacktesterEngine(initial_balance=10000)
45         metrics, history = engine.run(data_dict, strategy)
46
47         # Store Metrics
48         metrics['Strategy'] = name
49         results.append(metrics)
50
51         # Store Equity Curve (Portfolio Value)
52         # The engine stores it as 'TotalValue'
53         equity_curves[name] = history['TotalValue']
54
55     except Exception as e:
56         print(f"Failed to run {name}: {e}")
57
58 # --- Generate Outputs ---
59 os.makedirs("results", exist_ok=True)
60
61 # 1. Metrics CSV
62 results_df = pd.DataFrame(results)
63 # Reorder columns
64 cols = ['Strategy', 'Total Return', 'Sharpe Ratio', 'Max Drawdown',
65         'Geometric Mean Return', 'Annualized Volatility']
66 # Filter for cols that exist
67 cols = [c for c in cols if c in results_df.columns]
68 results_df = results_df[cols]
69
70 csv_path = "results/comparison_metrics.csv"
71 results_df.to_csv(csv_path, index=False)
72 print(f"Saved metrics to {csv_path}")
73 print(results_df)
74
75 # 2. Equity Curve Plot
76 from src.visualizer import Visualizer
77 # We assume Visualizer has a method for this, or we plot directly
78 here using matplotlib
79
80 plt.figure(figsize=(12, 6))
81 for name, series in equity_curves.items():
82     plt.plot(series.index, series.values, label=name)
83
84 plt.title("Strategy Comparison: Equity Curves")
85 plt.xlabel("Date")
86 plt.ylabel("Portfolio Value ($)")
87 plt.legend()

```

```

86     plt.grid(True)
87
88     plot_path = "results/equity_curves.png"
89     plt.savefig(plot_path)
90     print(f"Saved plot to {plot_path}")
91
92 if __name__ == "__main__":
93     run_comparison()

```

Listing A.12: Run Comparison Script

A.13 Data Download Script

Script to download historical data from Yahoo Finance:

```

1  """
2  Data Download Script
3  Downloads historical stock data from Yahoo Finance for the trading
4  strategies.
5
6  Usage:
7  python download_data.py
8  """
9
10 from src.data_collector import DataCollector
11
12 def main():
13     """Download historical data for AAPL, GOOGL, and MSFT."""
14     print("=" * 60)
15     print("DATA DOWNLOAD SCRIPT")
16     print("=" * 60)
17
18     # Initialize data collector
19     collector = DataCollector(data_dir="data")
20
21     # Define symbols to download
22     symbols = ["AAPL", "GOOGL", "MSFT"]
23
24     # Date range for the study (2015-2023)
25     start_date = "2015-01-01"
26     end_date = "2024-12-31"
27
28     print(f"\nDownloading data for: {' '.join(symbols)}")
29     print(f>Date range: {start_date} to {end_date}\n")
30
31     # Download data for all symbols
32     data_dict = collector.download_multiple_stocks(
33         symbols=symbols,
34         start_date=start_date,
35         end_date=end_date
36     )
37
38     print("\n" + "=" * 60)
39     print(f"Successfully downloaded data for {len(data_dict)} symbols")
40     print("=" * 60)
41
42     # Print summary
43     for symbol, df in data_dict.items():

```

```

42         print(f"{symbol}: {len(df)} rows, from {df.index[0]} to {df.
43             index[-1]}")
44 if __name__ == "__main__":
45     main()

```

Listing A.13: Data Download Script

A.14 Dependencies

Required Python packages (requirements.txt):

```

1 pandas>=2.0.0
2 numpy>=1.24.0,<2.0.0
3 matplotlib>=3.7.0
4 yfinance>=0.2.28
5 ta>=0.11.0
6 backtrader>=1.9.78
7 scikit-learn>=1.3.0
8 seaborn>=0.12.0
9 gymnasium>=0.29.0
10 stable-baselines3[extra]>=2.1.0
11 shimmy>=1.3.0
12 fpdf>=1.7.2

```

Listing A.14: Python Dependencies

A.15 Usage Instructions

A.15.1 Installation

```

1 # Clone the repository
2 git clone <repository_url>
3 cd AlgoTrading_Project
4
5 # Create virtual environment
6 python -m venv venv
7 source venv/bin/activate # Linux/Mac
8 # or: venv\Scripts\activate # Windows
9
10 # Install dependencies
11 pip install -r requirements.txt

```

Listing A.15: Installation Steps

A.15.2 Running the Backtest

```

1 # Download data first
2 python download_data.py
3
4 # Run single strategy
5 python main.py --strategy momentum --tickers AAPL,GOOGL,MSFT
6

```

```

7 # Run all strategies and generate comparison
8 python run_comparison.py

```

Listing A.16: Running Backtests

A.15.3 Training the TDQN Agent

```

1 # Train with default settings
2 python main.py --strategy tdqn --train --tickers AAPL,GOOGL,MSFT
3
4 # The model will be saved to models/tdqn_model.zip

```

Listing A.17: Training the AI Agent

A.16 Configuration Parameters

A.16.1 Strategy Parameters

Strategy	Parameter	Default Value
Momentum	j_months	12 (lookback period)
Momentum	k_months	3 (holding period)
Momentum	top_percentile	0.1 (top 10%)
Perfect Order	SMA periods	[10, 20, 50, 100, 200]
Perfect Order	ADX threshold	20
Perfect Order	confirmation days	5
Pairs Trading	window	20 (rolling window)
Pairs Trading	std_dev	2 (Z-score threshold)
RPMM	window	10 (lookback window)
TDQN	learning_rate	0.0001
TDQN	total_timesteps	10000

Table A.1: Default Strategy Parameters

A.16.2 Backtester Parameters

Parameter	Description
initial_balance	Starting capital (default: \$10,000)
commission_percent	Transaction cost as fraction (default: 0.001 = 0.1%)

Table A.2: Backtester Configuration

A.17 Output Files

A.17.1 Comparison Metrics CSV

Format of the output metrics file:

```
1 Strategy,Total Return,Sharpe Ratio,Max Drawdown,Geometric Mean Return,  
   Annualized Volatility  
2 Buy & Hold,6.12,1.01,-0.35,0.00087,0.25  
3 Momentum,3.75,0.81,-0.34,0.00069,0.26  
4 ...
```

Listing A.18: Metrics CSV Format

A.17.2 Equity Curves Plot

The equity curves visualization shows:

- X-axis: Date (2015-2023)
- Y-axis: Portfolio Value (\$)
- One line per strategy
- Legend identifying each strategy